

Máquinas conceituais e educação introdutória em programação

JUHA SORVA, Universidade Aalto

Este artigo reúne, resume e comenta diversas linhas de pesquisa que contribuíram para a nossa compreensão dos desafios enfrentados por programadores iniciantes ao aprenderem sobre a dinâmica de execução de programas e o papel do computador na execução de programas. Mais especificamente, a revisão abrange a literatura sobre concepções errôneas em programação, a teoria cognitiva de modelos mentais, a teoria construtivista do conhecimento e da aprendizagem, a pesquisa fenomenográfica sobre a experiência de programação e a teoria dos conceitos limiares. Esses conjuntos de trabalhos são examinados em relação ao conceito de uma “máquina conceitual” — um computador abstrato para executar programas de um tipo específico. De modo geral, a literatura aponta as máquinas conceituais como um grande desafio no ensino introdutório de programação. Argumenta-se que os instrutores devem reconhecer a máquina conceitual como um objetivo explícito de aprendizagem e abordá-la no ensino. O ensino dentro de alguns paradigmas de programação, como a programação orientada a objetos, pode se beneficiar do uso de múltiplas máquinas conceituais em diferentes níveis de abstração. São apresentadas algumas técnicas pedagógicas promissoras.

Categorias e Descritores de Assunto: K.3.2 **[Computadores e Educação]:** Educação em Ciência da Computação e Informação — *Educação em Ciência da Computação*

Termos gerais: Idiomas, Fatores Humanos, Teoria

Palavras-chave e frases adicionais: Máquina conceitual, introdução à programação, CS1, concepções errôneas, modelos mentais, construtivismo, fenomenografia, conceitos limiares, revisão de literatura

Formato de referência da ACM:

Sorva, J. 2013. Máquinas nocionais e educação introdutória em programação. *ACM Trans. Comput. Educ.*

13, 2, Artigo 8 (junho de 2013), 31 páginas.

DOI: <http://dx.doi.org/10.1145/2483710.2483713>

1. INTRODUÇÃO

Este artigo revisa a literatura sobre a aprendizagem da dinâmica de tempo de execução de programas de computador no contexto do ensino introdutório de programação. A revisão é estruturada em torno do conceito de uma “máquina conceitual”, um computador abstrato responsável pela execução de programas de um tipo específico. Utilizo o conceito de máquina conceitual para discutir a aplicabilidade da teoria da aprendizagem ao ensino introdutório de programação e como uma lente através da qual se pode analisar os resultados de pesquisas.

Uma Visão Multiparadigmática. Segundo o filósofo da ciência Charles Sanders Peirce, “o raciocínio não deve formar uma corrente que não seja mais forte do que seu elo mais fraco, mas um cabo cujas fibras podem ser tão finas quanto possível, desde que sejam suficientemente numerosas e intimamente conectadas” [Menand 1997, citando o original de 1868]. Mais recentemente, mas ainda nesse espírito pragmático, o caráter restritivo dos dogmas paradigmáticos rígidos sobre as abordagens e teorias de pesquisa tem sido reconhecido; o pluralismo paradigmático e a pesquisa com métodos mistos têm sido cada vez mais defendidos, tanto em geral quanto em outros aspectos.

Endereço do autor: J. Sorva, Departamento de Ciência da Computação e Engenharia, Universidade Aalto, Caixa Postal 15400, FI-00076 AALTO, Finlândia.

É concedida permissão para fazer cópias digitais ou impressas, totais ou parciais, desta obra para uso pessoal ou em sala de aula, sem custo, desde que as cópias não sejam feitas ou distribuídas para fins lucrativos ou comerciais e que esta nota seja exibida na primeira página ou na tela inicial, juntamente com a citação completa. Os direitos autorais dos componentes desta obra que pertencem a terceiros que não a ACM devem ser respeitados. É permitido fazer resumos com a devida atribuição.

Para copiar, republicar, postar em servidores, redistribuir para listas ou usar qualquer componente desta obra em outros trabalhos, é necessária permissão específica prévia e/ou o pagamento de uma taxa. As permissões podem ser solicitadas ao Departamento de Publicações da ACM, Inc., 2 Penn Plaza, Suite 701, Nova York, NY 10121-0701 EUA, fax +1 (212) 869-0481 ou permissions@acm.org. © 2013 ACM 1946-6226/2013/06-ART8 \$15,00 DOI: <http://dx.doi.org/10.1145/2483710.2483713>

[Morgan 2007; Tashakkori e Teddlie 2010] e, em particular, na pesquisa em educação computacional (CER) [Thota et al. 2012]. Quando as descobertas de diferentes tradições de pesquisa apontam na mesma direção, elas se reforçam mutuamente. Quando apontam em direções diferentes, nos dão material para reflexão e nos lembram que a aprendizagem é complexa e multifacetada. Frequentemente, diferentes perspectivas teóricas se complementam em vez de entrarem em conflito, e algo pode ser aprendido também com os pontos de conflito.

Esta revisão abrange trabalhos provenientes de múltiplas linhas de pesquisa em diversas disciplinas — psicologia, educação e computação. Essas linhas variam bastante em seus fundamentos epistemológicos, focos de pesquisa e metodologias típicas, mas todas contribuíram de diferentes maneiras para o nosso conhecimento sobre as dificuldades dos iniciantes com a dinâmica de programas e o conceito de máquina. Meu objetivo é reunir o conhecimento existente sobre esse tema e sintetizar um todo coerente a partir das diversas linhas de pesquisa.

Estrutura do artigo. A Seção 2 introduz o conceito de uma máquina conceitual, que as seções seguintes relacionam a diferentes (embora não independentes) áreas de pesquisa: estudos sobre concepções errôneas (Seção 3), teoria dos modelos mentais (Seção 4), construtivismo (Seção 5), pesquisa fenomenográfica sobre as diferentes maneiras pelas quais os aprendizes compreendem a programação (Seção 6) e a teoria dos conceitos limiares (Seção 7).

A Seção 8 considera as implicações pedagógicas da literatura para o ensino de máquinas nocionais aos alunos. A Seção 9 é dedicada ao papel das máquinas nocionais na programação orientada a objetos. A Seção 10 contém algumas considerações finais.

Este artigo, que se concentra em máquinas conceituais, foi editado a partir de uma revisão bibliográfica mais extensa sobre pesquisas em educação introdutória em programação, que aparece na tese de doutorado do autor [Sorva 2012]. Os fundamentos filosóficos das teorias de aprendizagem que aparecem neste artigo — e as tensões entre elas — também são discutidos nessa obra. Partes das Seções 7 e 8 foram originalmente publicadas em um artigo de conferência [Sorva 2010].

2. MÁQUINAS CONCEITUAIS — O QUE SÃO ELAS?

O termo *máquina conceitual* foi introduzido na CER por Benedict du Boulay, que o usou para se referir às “propriedades gerais da máquina que se está aprendendo a controlar” à medida que se aprende a programar [du Boulay 1986]. Uma máquina conceitual é um computador idealizado “cuja propriedades são implícitas pelas construções na linguagem de programação empregada” [du Boulay 1986], mas que também pode ser explicitada no ensino [du Boulay et al. 1981].

As abstrações são criadas com um propósito; o propósito de uma máquina conceitual é explicar a execução de programas. Uma máquina conceitual é uma caracterização do computador em seu papel como executor de programas em uma linguagem específica ou em um conjunto de linguagens relacionadas.

Uma máquina conceitual engloba capacidades e comportamentos de hardware e software que são abstratos, mas suficientemente detalhados, para um determinado contexto, para explicar como os programas são executados e qual a relação dos comandos da linguagem de programação com essas execuções. A seguinte citação de Bruce-Lockhart e Norvell [2007] ilustra bem essa ideia.

Enquanto nos esforçávamos para transmitir aos nossos alunos que cada instrução que escreviam tinha significado, tivemos uma percepção importante. A máquina (ou sistema) T que estávamos programando (e que queríamos que os alunos entendessem) não era realmente um computador, pelo menos não no sentido clássico de hardware. Considere o seguinte código C simples:

```
int x = 5;
int y = 12; int z;
z = y/5
+ 3.1;
```

Na linguagem de programação, dizemos que há quatro instruções a serem executadas. Instruções para *quê* e a serem executadas por *quem*? T, é claro, mas T certamente não é a CPU. As três primeiras "instruções" são, na verdade, para o compilador... A quarta é um campo minado. Há um truncamento e duas conversões automáticas de tipo...

T é definido, pelo menos em parte, pela linguagem.

No caso das linguagens C++ e Java, T é uma abstração que combina aspectos do computador, do compilador e do esquema de gerenciamento de memória. Nosso T não é tão "conhecível" [quanto alguns outros sistemas]. Isso não nos exime da responsabilidade de ao menos tentar defini-lo.

2.1. Muitas Máquinas

Como uma máquina conceitual está ligada a uma forma de programação, diferentes tipos de linguagens de programação terão máquinas conceituais diferentes. Uma máquina conceitual orientada a objetos em Java pode ser bastante diferente de uma máquina conceitual funcional em Lisp. A maioria das máquinas conceituais que executam Prolog provavelmente também serão bastante diferentes. Linguagens semelhantes podem estar associadas a máquinas conceituais semelhantes ou até mesmo idênticas. Algumas máquinas conceituais podem não ser muito "parecidas com máquinas" se forem baseadas, por exemplo, em matemática e cálculo lambda.

Não apenas existem diferentes máquinas conceituais para diferentes linguagens e paradigmas, mas até mesmo uma única linguagem pode ser associada a diferentes máquinas conceituais. Afinal, não existe uma abstração única do computador para descrever a execução de programas em uma linguagem. Consideremos, por exemplo, as seguintes maneiras de entender a execução de programas em Java.

Uma máquina conceitual para programas Java de thread única poderia definir o comportamento do computador em tempo de execução em termos de áreas de memória abstratas, como a pilha de chamadas e o heap, e regras de fluxo de controle associadas às instruções do programa. Essa máquina conceitual incorpora ideias como "o computador é capaz de rastrear variáveis com nomes diferentes, cada uma podendo ter um único valor", "um frame na pilha de chamadas contém parâmetros e outras variáveis locais", "o computador percorre as linhas do programa em ordem, exceto quando encontra uma instrução que o faz saltar para uma linha diferente", e assim por diante. Uma máquina conceitual Java em um nível de abstração mais alto poderia definir o computador como um dispositivo capaz de rastrear objetos que foram criados e trocar mensagens entre esses objetos, conforme instruído pelas chamadas de método em um programa Java. Os objetos se revezam na execução de seus comportamentos definidos e param para esperar por outros objetos cujos métodos eles chamam. O computador armazena os objetos e garante que cada objeto tenha sua "vez de agir" quando apropriado. Uma terceira máquina conceitual Java poderia definir o papel do computador em um nível de abstração relativamente baixo em termos de bytécodes e dos componentes da Máquina Virtual Java.

2.2. Características das Máquinas Conceituais

Em resumo, uma máquina conceitual: —é

uma abstração idealizada do hardware do computador e de outros aspectos do ambiente de execução dos programas;

—serve para o propósito de compreender o que acontece durante a execução do programa; — está associado a um ou mais paradigmas ou linguagens de programação, e possivelmente com um ambiente de programação específico;

—permite a semântica do código de programa escrito nesses paradigmas ou linguagens (ou subconjuntos dos mesmos) a serem

descritos; —oferece uma perspectiva particular à execução de programas; e — reflete corretamente o que os programas fazem quando executados.

Também é instrutivo considerar o que uma máquina conceitual *não* é, conforme eu uso o termo.

Uma máquina conceitual não é uma representação mental que um aluno tem do computador, ou seja, a noção que alguém tem da máquina. Os alunos, no entanto, formam modelos mentais de máquinas conceituais, como discutido no texto a seguir.

Uma máquina conceitual não é uma descrição ou visualização do computador, embora descrições e visualizações de uma máquina conceitual possam ser criadas por professores para alunos, por exemplo.

Por fim, uma máquina conceitual geralmente não é uma abstração geral, independente de linguagem e paradigma, do computador. Embora algumas máquinas conceituais sejam genéricas o suficiente para abranger muitas linguagens, um paradigma de programação inteiro ou até mesmo todas as linguagens de programação, a máquina conceitual prototípica se limita a uma única linguagem ou a algumas linguagens semelhantes. Da perspectiva de um curso monolíngue típico de Introdução à Ciência da Computação, do programador iniciante monolíngue e da presente revisão, são as máquinas conceituais específicas de paradigma e linguagem que geralmente despertam maior interesse.

As próximas cinco seções relacionam linhas de pesquisa dentro da CER ao conceito de máquina conceitual. O foco da literatura existente é a programação imperativa (seja procedural ou orientada a objetos), assim como o da minha revisão. A programação imperativa orientada a objetos é um paradigma dominante em Ciência da Computação para iniciantes (CS1), dentro do qual existem diversas maneiras de pensar sobre máquinas conceituais, o que é discutido separadamente na Seção 9. Não abordo o "terceiro paradigma de CS1", a programação funcional; os resultados empíricos e as implicações pedagógicas discutidos não se aplicam necessariamente à programação funcional/declarativa.

3. MÁQUINAS CONCEITUAIS E A LITERATURA SOBRE CONCEITOS ERRÔNEOS

"Em duas ocasiões me perguntaram: 'Por favor, Sr. Babbage, se o senhor inserir números errados na máquina, as respostas corretas sairão?'..." Não sou capaz de compreender adequadamente o tipo de confusão de ideias que poderia provocar tal questão" [Babbage 1864].

Em algumas disciplinas, conceitos e fenômenos são amplamente negociáveis e passíveis de interpretação. Os alunos podem ser incentivados a interpretar as coisas de maneira pessoal e a desenvolver estruturas conceituais alternativas. Certamente, muitos conceitos da computação também são assim. No entanto, a computação também apresenta muitos conceitos que são precisamente definidos e implementados dentro de sistemas técnicos. Espera-se que os alunos cheguem a maneiras particulares de entender o que a instrução `assignment` em Java faz, o que é um objeto e como um determinado programa em C é executado. Às vezes, um programador iniciante "não entende" um conceito ou "o interpreta incorretamente" de uma maneira que não é uma interpretação alternativa inofensiva (ou desejável). Compreensões incorretas e incompletas de conceitos de programação resultam em comportamento improdutivo de programação e programas disfuncionais.

Infelizmente, concepções errôneas até mesmo dos conceitos de programação mais fundamentais, que são triviais para especialistas, são comuns entre os novatos e difíceis de superar. Estudos recentes que medem o conhecimento conceitual dos alunos sugerem que os cursos introdutórios de programação não são particularmente eficazes no ensino de conceitos fundamentais e que os problemas não se limitam a uma única instituição nem são causados pelo uso de uma linguagem de programação específica [Elliott Tew 2010; Kunkle 2010].

3.1. Evidências de Conceitos Errôneos

Nas últimas décadas, muitos pesquisadores relataram as dificuldades que os alunos de programação enfrentam com conceitos fundamentais e os tipos de entendimentos incompletos e incorretos que esses alunos demonstram. Uma visão geral desse trabalho aparece no texto a seguir. Um catálogo mais detalhado de conceitos errôneos específicos

Os dados relatados na literatura podem ser encontrados na tese de doutorado na qual o presente artigo se baseia [Sorva 2012].

Uso aqui a palavra "concepção errônea" em um sentido muito amplo para me referir a entendimentos que são deficientes ou inadequados para muitos contextos práticos de programação. O que agrupei como "concepções errôneas", os pesquisadores originais denominaram de diversas maneiras: "concepções errôneas", "compreensões parciais", "compreensões incorretas", "regras construídas pelos alunos", "dificuldades", "erros", "falhas" e assim por diante.¹

Primeiros trabalhos sobre programação imperativa. Bayman e Mayer [1983] estudaram as interpretações de instruções na linguagem BASIC por iniciantes, pedindo aos alunos que escrevessem explicações em inglês simples de programas. Eles listaram uma série de concepções errôneas sobre a semântica do BASIC; por exemplo, instruções LET são entendidas como armazenamento de equações em vez de atribuição a uma variável. Quase na mesma época, Soloway, Bonar e seus colegas também exploraram concepções errôneas e erros de iniciantes e discutiram como eles podem ser causados por conhecimento externo à programação, particularmente por analogias com a semântica cotidiana da linguagem natural [Soloway et al. 1982; Bonar e Soloway 1985; Soloway et al. 1983; du Boulay 1986].

Samurçay [1989] estudou as respostas que iniciantes em programação deram a três tarefas de conclusão de programas e relatou que a inicialização de variáveis, em particular, era difícil para os alunos compreenderem. Putnam, Sleeman, Baxter e Kuspa [Putnam et al. 1986; Sleeman et al. 1986] analisaram as respostas dos alunos a testes de compreensão de código e entrevistas subsequentes. Eles listam inúmeros erros — superficiais e profundos — que os alunos cometem com variáveis, atribuição, instruções de impressão e fluxo de controle.

Passagem de Parâmetros e Recursão. Fleury [1991] e Madison e Gifford [1997] entrevistaram e observaram estudantes para descobrir várias concepções de passagem de parâmetros. Seus resultados sugerem que mesmo estudantes que às vezes são capazes de produzir código funcional que utiliza parâmetros podem interpretar os conceitos envolvidos de maneiras diferentes.

Kahney [1983] descobriu que os alunos têm vários modelos falhos de recursão, como o "modelo de loop", no qual a recursão é entendida como sendo muito semelhante à iteração. O trabalho de Kahney foi posteriormente aprofundado por diversos autores [Bhuiyan et al. 1990; George 2000a; 2000b; Gotschi et al. 2003]. A recursão também é um dos fenômenos investigados por Booth em seu trabalho fenomenográfico sobre o aprendizado de programação [Booth 1992]. Booth identificou três maneiras diferentes de vivenciar a recursão: como um constructo de programação, como um meio de repetição e como uma autorreferência; os alunos nem sempre conseguem compreender todos esses aspectos.

Temas recentes: POO e Java. Desde a década de 1990, o interesse em CER (Concepção, Relevância e Equilíbrio) tem se deslocado da programação procedural para a programação orientada a objetos. Diversos estudos relataram maneiras pelas quais os alunos interpretam erroneamente os conceitos e recursos da orientação a objetos em linguagens OO. Holland et al. [1997] observaram várias concepções errôneas que os alunos têm sobre objetos. Por exemplo, os alunos às vezes confundem os conceitos de objeto e classe, e podem confundir uma variável de instância chamada nome com a identidade do objeto. Mais concepções errôneas de iniciantes sobre POO foram relatadas por Detienne [1997] como parte de sua revisão das consequências cognitivas da abordagem orientada a objetos no ensino de programação.

Fleury [2000] relatou que os alunos criam suas próprias regras, desnecessariamente rígidas, sobre o que acontece nos programas e o que funciona na programação Java. Por exemplo, alguns

¹Algumas dessas diferenças de terminologia são superficiais, outras são motivadas pela teoria da aprendizagem — muitos fenomenógrafos, por exemplo, adotam uma perspectiva na qual formas mais pobres de compreender conceitos podem ser vistas como versões limitadas (em vez de equivocadas) de compreensões mais ricas (ver Seção 6).

Dos alunos que ela estudou, alguns acreditavam que o operador ponto só podia ser aplicado a métodos e que o único propósito de um construtor era inicializar variáveis de instância.

Hristova et al. [2003] listam uma série de erros comuns que os alunos cometem ao programar em Java. Muitos deles são de natureza sintática superficial (por exemplo, confundir um operador com outro), mas alguns sugerem equívocos mais profundos. Ragonis e Ben-Ari [2005a] relatam os resultados de um estudo de pesquisa-ação de longo prazo e amplo escopo com alunos do ensino médio aprendendo programação orientada a objetos. Eles descobriram uma gama impressionante de equívocos e outras dificuldades que os alunos têm com os conceitos de orientação a objetos, a linguagem Java e o ambiente de programação BlueJ.

Teif e Hazzan [2006] observaram alunos de programação introdutória em dois cursos do ensino médio e discutiram a confusão conceitual dos alunos em relação a classes e objetos. Por exemplo, os alunos podem pensar erroneamente que a relação entre uma classe e suas instâncias é partonômica, ou seja, que os objetos são partes de uma classe. Eckerdal e Thune [2005] também estudaram as concepções que os alunos têm desses conceitos fundamentais da orientação a objetos. Seus resultados destacam o fato de que nem todos os alunos aprendem a apreciar objetos e classes como entidades dinâmicas em tempo de execução ou como ferramentas de modelagem que representam aspectos de um domínio de problema.

Vainio [2006] usou entrevistas para obter a compreensão dos alunos sobre conceitos de programação e relata uma série de concepções errôneas sobre conceitos fundamentais, por exemplo, a ideia de que o tipo de um valor pode mudar dinamicamente (em Java).

Diversos relatórios complementares confirmaram a existência de uma série de compreensões incorretas sobre atribuição, variáveis e as relações entre objetos e variáveis [Ma 2007; Sorva 2007; 2008; Doukakis et al. 2007]. Por exemplo, Ma aplicou um teste com questões abertas e de múltipla escolha sobre atribuição em Java a um grande número de alunos voluntários do primeiro ano de Ciência da Computação. Ele analisou os resultados qualitativa e quantitativamente para compreender os modelos mentais dos alunos sobre atribuição e semântica de referência.

Sajaniemi et al. [2008] investigaram a compreensão que programadores iniciantes têm do estado do programa, pedindo a alunos de CS1 que desenhasssem e escrevessem sobre como percebiam o estado de determinados programas Java em estágios específicos de execução. Eles descobriram inúmeras concepções errôneas relacionadas à passagem de parâmetros e a conceitos de orientação a objetos.

Como parte de um projeto para desenvolver um inventário de conceitos para CS1, Kaczmarczyk et al. [2010] entrevistaram alunos para identificar concepções errôneas. Quatro temas foram identificados: a relação entre elementos da linguagem e memória, laços while, o conceito de objeto e a capacidade de rastreamento de código.

Dois estudos recentes revelaram ainda mais equívocos sobre programas Java orientados a objetos. As entrevistas com estudantes realizadas por Chen et al. [2012] sugeriram diversas formas de confusão conceitual (por exemplo, entre tipos definidos pelo usuário e tipos predefinidos, e membros estáticos e membros constantes). Shmallo et al. [2012] utilizaram questionários e entrevistas para produzir uma lista de equívocos que ou expandem demais as definições (por exemplo, “um identificador pode se referir a dois ou mais objetos”) ou restringem incorretamente as propriedades das construções (por exemplo, “é impossível acessar um atributo estático através do nome da classe correspondente”).

3.2. Conceitos errôneos sobre a máquina

oculta. As “mudanças internas ocultas” que ocorrem dentro da máquina (teórica) têm sido apontadas como problemáticas para iniciantes [du Boulay 1986]. Considere, por exemplo, a noção de que a atribuição de objeto $a=b$ (em Java) copia os valores dos atributos de instância de um objeto para outro objeto. Superar esse mal-entendido requer o conceito de referência a um objeto, algo que não é aparente no código.

Muitos equívocos, senão a maioria deles, têm a ver com aspectos que não são facilmente compreendidos.

Visíveis, mas ocultos no mundo de tempo de execução da máquina conceitual: referências, objetos, atualizações automáticas de variáveis de controle de loop e assim por diante.

Aspectos sutis e "ocultos" dos programas também estão no topo de várias pesquisas sobre tópicos difíceis de CS1. Milne e Rowe [2002] pesquisaram as opiniões de alunos e tutores sobre a dificuldade de conceitos de programação. Eles concluíram que os conceitos mais difíceis, como ponteiros, têm a ver com o uso da memória em tempo de execução e que "esses conceitos são difíceis apenas devido à incapacidade do aluno de compreender o que está acontecendo com seu programa na memória, pois ele é incapaz de criar um modelo mental claro de sua execução". Uma pesquisa Delphi com educadores da área de computação identificou referências, ponteiros e um modelo geral de memória como alguns dos tópicos mais difíceis em programação introdutória [Goldman et al. 2008]. Os alunos de diversas instituições de ensino pesquisados por Lahtinen et al. [2005] consideraram ponteiros e recursão os tópicos mais difíceis.

Capacidades do Computador e o Superbug. Algumas concepções errôneas genéricas sobre o computador e a natureza dos programas podem estar por trás de muitas outras concepções errôneas mais específicas que foram descobertas. Pea [1986] sugeriu, com base em sua análise de bugs de programadores iniciantes, que muitos deles têm origem em um "superbug", ou seja, na suposição de que existe uma mente inteligente oculta dentro do computador que ajuda o programador a atingir seus objetivos. Evidências de expectativas irrealistas sobre as capacidades de raciocínio do computador também podem ser vistas na literatura mais recente; por exemplo, os alunos estudados por Ragonis e Ben-Ari [2005a] às vezes esperavam que o computador pudesse tirar conclusões do contexto lógico de declarações ou da semântica do mundo real de identificadores.

Não é apenas o que o computador faz nos bastidores que precisa ser compreendido. O novato também precisa compreender o que a máquina conceitual *não* faz, a menos que seja especificamente instruído pelo programador. As pessoas não descrevem processos naturalmente da maneira que os programadores precisam. Os equivalentes das cláusulas "else", por exemplo, são notavelmente ausentes nas descrições de processos feitas por não programadores, já que as pessoas tendem a se esquecer de ramificações alternativas e podem considerá-las óbvias demais para merecerem atenção [Miller 1981; Pane et al. 2001]. O novato precisa aprender o que a máquina conceitual faz por ele, por um lado, e qual é a sua própria responsabilidade como programador, por outro.

Resumo da Seção. A literatura sobre concepções errôneas sugere que muitos dos problemas de programadores iniciantes estão relacionados a uma compreensão inadequada da máquina conceitual, especialmente dos processos "ocultos" que não são diretamente aparentes no código do programa. Alguns desses problemas são específicos de como a máquina lida com construções individuais, enquanto outros são mais genéricos e envolvem concepções errôneas mais amplas da máquina conceitual e suas capacidades.

4. MÁQUINAS NOCIONAIS E TEORIA DOS MODELOS MENTAIS

O conceito teórico de modelo mental tem sido utilizado na literatura de Pesquisa Comparativa de Eventos (CER, na sigla em inglês) para explicar (entre outras coisas) como as pessoas percebem a máquina conceitual e como rastreiam programas mentalmente. Esta seção revisa e integra esses dois aspectos.

4.1. Uma Introdução aos Modelos Mentais

Um *modelo mental* é uma estrutura mental que representa algum aspecto do ambiente de uma pessoa. De particular interesse para a maioria dos teóricos de modelos mentais têm sido as interações entre humanos e sistemas causais, como circuitos elétricos, mecanismos de controle de processos e software [Schumacher e Czerwinski 1992]. Postula-se que as pessoas utilizam

modelos mentais de tais sistemas para descrever o propósito e os mecanismos subjacentes dos sistemas para si mesmos e para prever estados futuros do sistema.²

Características típicas. De acordo com a descrição seminal de Norman [1983], os modelos mentais:

- refletem as crenças das pessoas sobre os sistemas que utilizam e sobre as suas próprias limitações, e incluem declarações sobre o grau de incerteza que as pessoas sentem em relação a diferentes aspectos do seu conhecimento;
 - Fornecem explicações parcimoniosas e simplificadas de fenômenos complexos; —Frequentemente contêm apenas descrições incompletas e parciais de operações, podendo apresentar grandes áreas de incerteza; —São “não científicas” e imprecisas, e muitas vezes baseadas em palpites, suposições e crenças ingênuas, bem como em regras “supersticiosas” que “parecem funcionar”, mesmo que não façam sentido;
 - geralmente são deficientes de diversas maneiras, talvez incluindo contraditórias, errôneas-conceitos novos e desnecessários;
 - faltam limites firmes, de modo que pode não ficar claro para a pessoa exatamente quais aspectos ou partes de um sistema seu modelo abrange — mesmo nos casos em que o modelo está completo e correto;
 - evoluem ao longo do tempo à medida que as pessoas interagem com os sistemas e modificam seus modelos para obter resultados viáveis; —
 - estão sujeitos a mudanças a qualquer momento; e —
 - podem ser “executados” para simular e prever mentalmente o comportamento do sistema, embora as pessoas
- A capacidade de executar modelos é limitada.

É comum as pessoas confundirem e combinarem modelos mentais de sistemas semelhantes. Uma pessoa também pode ter múltiplos modelos mentais de um mesmo sistema. Esses múltiplos modelos podem abranger diferentes partes do sistema de forma não sobreposta e complementar, ou podem ser modelos paralelos — talvez até contraditórios — das mesmas partes.

Norman [1983] sugeriu que as pessoas se baseiam em seus modelos mentais para desenvolver padrões de comportamento que as fazem sentir mais seguras sobre como interagem com os sistemas, mesmo quando sabem que o que estão fazendo não é necessário. Os modelos mentais precisam ser minimamente viáveis para serem mantidos e nem precisam ser precisos para que alguns usuários do sistema se sintam totalmente satisfeitos — uma abordagem de “ignorância é uma bênção” [Westbrook 2006].

O Desenvolvimento da Especialização. Embora os modelos mentais sejam claramente úteis, eles também são potencialmente perigosos. Um modelo mental impreciso levará a erros. A pesquisa de Kempton [1986] contribui com o exemplo do termostato: modelos mentais baseados em analogias inadequadas, digamos, com aceleradores de carros, levam as pessoas a pensar que aumentar a potência do termostato ao máximo aquecerá a casa mais rapidamente. Um modelo mental inadequado de um ambiente de programação de computador resultará em erros. Mesmo que as próprias pessoas não exijam que seus modelos mentais sejam completos e precisos para serem usados, elas “certamente funcionam com diferentes níveis de eficiência e eficácia ao empregarem modelos mentais imprecisos e/ou incompletos” [Westbrook 2006].

A qualidade dos modelos mentais tem sido apontada como um dos fatores que diferenciam o especialista do novato. Os especialistas se baseiam em analogias fundamentadas em modelos mentais preexistentes ao se depararem com novas situações que exigem a formação de novos modelos — assim como os novatos. Contudo, os modelos mentais dos especialistas são robustos, baseados em uma compreensão criteriosa do sistema.

²Nesta revisão, quando escrevo sobre modelos mentais, refiro-me apenas aos chamados modelos *mentais causais* — modelos mentais de longo prazo de sistemas causais. Outro corpo de pesquisa, menos pertinente para os propósitos atuais, preocupa-se com *modelos mentais lógicos* [Johnson-Laird 1983; Markman e Gentner 2001].

componentes e permitem lidar com situações imprevistas. Como a incerteza sobre as capacidades do sistema pode levar à tentativa de múltiplas abordagens, os novatos tendem a confiar mais em múltiplos modelos mentais causais inconsistentes dos sistemas, enquanto os especialistas são menos propensos a fazê-lo. Comparadas aos modelos ingênuos ad hoc frequentemente empregados por novatos, as representações mentais dos especialistas são relativamente estáveis como resultado de longa experiência [Schumacher 1987; Schumacher e Czerwinski 1992; Gentner e Stevens 1983].

Aprendendo sobre um Sistema. Os modelos mentais muitas vezes não são produto de raciocínio deliberado; eles podem ser formados intuitivamente e de forma bastante inconsciente. O conhecimento prévio desempenha um papel fundamental nos estágios iniciais de um modelo mental de um sistema, visto que a formação do modelo frequentemente se baseia em metáforas e analogias [Gentner e Gentner 1983; Schumacher e Czerwinski 1992]. Para criar modelos iniciais de sistemas desconhecidos, as pessoas geralmente resgatam experiências com sistemas superficialmente semelhantes (por exemplo, outros sistemas com uma interface gráfica de usuário similar). Com a experiência, a compreensão das relações causais emerge por meio da exposição prolongada ao sistema, mas é irrealista esperar a transferência de um modelo mental antes que ele esteja bem consolidado. De acordo com Schumacher e Gentner [1988], quanto menos semelhantes superficialmente dois sistemas forem, pior será a transferência, mesmo quando os sistemas são funcionalmente isomórficos.

Do ponto de vista da aprendizagem, o problema reside no fato de que, embora os modelos possam ser desenvolvidos ou corrigidos por meio da prática e da instrução, as pessoas tendem a se apegar a modelos existentes, emocionalmente confortáveis e familiares. Moray, por exemplo, descobriu que mudar o modelo mental de alguém levava significativamente mais tempo do que o necessário para formar um modelo inicial de complexidade semelhante [Schumacher e Czerwinski 1992]. Para piorar a situação, meras coincidências podem reforçar a confiança das pessoas em seus modelos existentes, ainda que falhos [Besnard et al. 2004].

4.2. Modelos da Máquina

O aluno de CS1 precisa construir um modelo mental de uma máquina conceitual para programa.

Muitos autores discutiram o papel da máquina na programação introdutória em termos de modelos mentais. Alguns "atribuem o conhecimento frágil dos alunos sobre programação em grande parte à falta de um modelo mental do computador" [Perkins et al. 1990].

Alega-se que as dificuldades dos novatos no desenvolvimento e depuração de seus programas decorrem do fato de que "seu modelo mental de como o computador funciona é inadequado" [Smith e Webb 1995]. "É amplamente aceito que a programação requer acesso a algum tipo de 'modelo mental' do sistema" [Canas et al. 1994].

Um modelo mental de uma máquina conceitual — juntamente com a compreensão do programa (ver discussão posterior) — permite que um programador faça inferências sobre o comportamento do programa e preveja mudanças futuras nos programas que está escrevendo. Um iniciante terá apenas um modelo mental do sistema específico que está usando para programar (uma máquina conceitual específica dependente da linguagem), mas à medida que ganha experiência, ele forma modelos mentais de outras máquinas conceituais e esquemas cada vez mais gerais do comportamento do computador.

A teoria dos modelos mentais nos diz que as pessoas frequentemente usam analogias baseadas em características superficiais ao formar modelos mentais de novos sistemas que encontram. Há evidências disso também no ensino de programação. As propriedades da máquina estão implícitas nas construções da linguagem de programação correspondente — a fachada visível da máquina conceitual. De fato, o código do programa é uma base fértil para a construção de um modelo mental, já que "os novatos fazem inferências sobre a máquina conceitual a partir dos nomes das instruções" [du Boulay et al. 1981]. Superficialmente, muitas linguagens de programação se assemelham à linguagem natural e à linguagem da matemática. Concepções errôneas (ver Seção 3) são causadas por analogias malsucedidas com esses domínios, como

quando os alunos concluem que a instrução Java $a=b+1$ define uma equação matemática.

O modelo mental de um novato sobre uma máquina hipotética tende a ser — como é típico dos modelos mentais em geral — incompleto, pouco científico, deficiente, sem limites claros e sujeito a mudanças a qualquer momento. Pode ser baseado em palpites que se fundamentam em características superficiais do programa, como palavras-chave e identificadores. Apesar dessas limitações, o aprendiz pode se sentir confortável com o modelo e confiar nele ao desenvolver padrões de comportamento para programação. Novatos também podem usar múltiplos modelos, possivelmente contraditórios, para lidar com diferentes situações. Instruções de atribuição com variáveis inteiras podem ser explicadas por um modelo mental, por exemplo, e atribuições usando tipos de registro por um modelo completamente diferente.

Em contraste, os modelos mentais de especialistas são mais estáveis e precisos, e baseiam-se em princípios gerais em vez de características superficiais. Um desafio do ensino de programação é facilitar a evolução dos modelos dos alunos para que possuam essas características. Ensinar sobre uma máquina conceitual explícita pode diminuir o nível de liberdade que os alunos se permitem ao formar modelos mentais e pode resultar em modelos melhores (ver Seção 8). Auxiliar a formação de modelos mentais o mais cedo possível é importante, pois mudar um modelo mental arraigado, porém falho, é mais difícil do que ajudar a construir um modelo desde o início. Pesquisas sobre modelos mentais preveem ainda que programadores iniciantes podem ter dificuldades em transferir seus modelos mentais de uma máquina conceitual para uma linguagem de programação, mesmo que superficialmente diferente, a menos que o modelo original esteja bem consolidado por meio de uma quantidade substancial de prática.

4.3. Modelos Mentais e Rastreamento

"Para entender um programa, você deve se tornar tanto a máquina quanto o programa" [Perlis 1982].

A teoria dos modelos mentais também tem sido aplicada ao ensino introdutório de programação. ao considerar o papel dos modelos mentais no rastreamento de programas.

Executando um Modelo Mental. De acordo com a descrição de Norman, os modelos mentais são "executáveis". Ou seja, as pessoas podem usar modelos mentais para raciocinar sobre sistemas em situações específicas, para visualizar mentalmente como um sistema funciona e para prever o comportamento e os estados de um sistema, dado um conjunto de condições iniciais [Gentner e Stevens 1983; Markman e Gentner 2001]. Por exemplo, as pessoas podem executar seus modelos mentais para prever as trajetórias de bolas colidindo em um sistema físico, o comportamento de um sistema de software existente sob determinados parâmetros ou o comportamento de um programa de computador que estão projetando.

A simulação mental é realizada na *memória de trabalho* — o principal gargalo do aparato cognitivo humano [Tuovinen 2000]. Frequentemente envolve imagens visuais e pode ter um componente motor. Como a capacidade da memória de trabalho é muito limitada, não surpreende que pesquisadores tenham descoberto que as simulações mentais envolvem apenas um número muito pequeno de fatores. De acordo com Klein [1999], por exemplo, mesmo as simulações mentais de especialistas raramente envolvem mais de três fatores (ou "partes móveis") e seis estados de transição (estágios). Simular o comportamento de um sistema em um baixo nível de abstração pode falhar devido ao excesso de variáveis ou estados. A simulação em um nível de abstração excessivamente alto também não produzirá soluções funcionais para os problemas e, mesmo quando o nível geral de abstração é apropriado, as pessoas tendem a negligenciar ou abstrair informações importantes. Para resolver problemas com sucesso, é crucial simular sistemas em um nível de abstração adequado ao problema em questão e focar exatamente nos fatores importantes para produzir o tipo de previsão ou solução desejada. Fazer isso é difícil e requer experiência.

Rastreado o Programa e a Máquina. No contexto da programação, simulações da execução de programas são frequentemente chamadas de *rastreamento*. O rastreamento é uma habilidade fundamental de programação que programadores experientes usam rotineiramente durante tarefas de projeto e compreensão [Adelson e Soloway 1985; Soloway 1986; Detienne e Soloway 1990].

Pode-se considerar o rastreamento de programas como a execução de um modelo mental de um programa com alguma entrada, enquanto se executa também um modelo mental separado de uma máquina hipotética para a qual o próprio programa serve como entrada. No entanto, quando executamos um programa, “o computador efetivamente se torna o mecanismo descrito pelo programa” [du Boulay 1986]. Quando falamos de execução de programas, usamos expressões como “o programa faz X” e “o computador faz X” para significar, na prática, a mesma coisa. Como mencionado anteriormente, as fronteiras dos modelos mentais são frequentemente vagas. Embora os modelos da máquina e do programa possam ser vistos analiticamente como separados, eles podem estar inextricavelmente entrelaçados durante o rastreamento mental e não são necessariamente distintos na memória do programador.

A seguir, descrevo o rastreamento mental como a execução de um único modelo mental que engloba tanto a máquina conceitual quanto o programa que está sendo rastreado.

Rastreamento e Iniciantes. Programadores preferem o rastreamento simbólico, que utiliza valores não específicos — o que é típico de simulações mentais em geral. Especialistas tendem a usar o rastreamento concreto apenas em situações desafiadoras, onde outros métodos falham. Iniciantes, que não percebem os tipos de padrões no código que permitem a abstração, e cujos programas são mais propensos a erros do que os de especialistas, frequentemente precisam de rastreamento concreto.

Infelizmente, programadores iniciantes costumam ter muita dificuldade com o rastreamento de código.

Em um estudo bastante conhecido, Lister et al. [2004] avaliaram a capacidade dos alunos de acompanhar a execução de um programa. Eles aplicaram um questionário de múltipla escolha a estudantes de diversas instituições de ensino ao redor do mundo. As questões exigiam que os alunos, que estavam perto de concluir o primeiro ano da disciplina de Ciência da Computação (CS1), previssem os valores de variáveis em pontos específicos da execução e completassem pequenos programas inserindo uma linha de código escolhida dentre várias alternativas fornecidas. Lister et al. concluíram que muitos alunos não conseguiram acompanhar a execução do programa. Embora obviamente houvesse alguma variação na capacidade dos alunos entre as instituições, os resultados foram decepcionantes em geral.

Outros estudos produziram resultados semelhantes. Por exemplo, um estudo multi-institucional anterior, realizado por Sleeman et al. [1986], analisou entrevistas baseadas em código e concluiu que “pelo menos metade dos alunos *não* conseguia rastrear programas sistematicamente” quando solicitado, e em vez disso, “frequentemente decidiam o que o programa faria com base em algumas instruções-chave”. Adelson e Soloway [1985] descobriram que os iniciantes eram incapazes de rastrear mentalmente as interações dentro do sistema que eles mesmos estavam projetando. Kacz-Marczyk et al. [2010] relatam a incapacidade de “rastrear o código linearmente” como um tema importante das dificuldades dos iniciantes. Uma série de estudos recentes indicou que muitos alunos não conseguem entender a sequência de instruções a ponto de não conseguirem compreender uma simples troca de valores de variáveis em três linhas [Corney et al. 2011; Simon 2011; Teague et al. 2012; Murphy et al. 2012]. Em um estudo, o problema persistiu mesmo entre alunos que cursavam uma disciplina de programação de nível avançado [Simon 2011].

Além disso, Perkins et al. [1986] descobriram que muitos novatos nem sequer tentam rastrear os programas que escrevem, mesmo quando precisam fazê-lo para progredir. Em seu estudo, “os alunos raramente rastream seus programas sem serem incentivados”. A relutância em rastrear os próprios programas, argumentam Perkins et al., pode ser devida a razões como a falta de percepção da importância do rastreamento, a falta de confiança na própria capacidade de rastreamento, a falta de compreensão da linguagem de programação ou o foco na saída do programa em vez do que acontece internamente.

Representações de Estado. Rastrear um programa exige que o programador acompanhe o estado de execução do programa, ou seja, simule o trabalho da máquina hipotética. Uma descrição do estado do programa, que Perkins et al. [1986] chamam de *representação de estado*, deve

Ser dinâmico, mudando conforme a execução prossegue. Uma representação de estado é baseada nos elementos do modelo mental da máquina conceitual utilizada: variáveis, objetos, referências, ativações de funções e assim por diante. Uma representação de estado de um programa complexo envolve uma quantidade de informações que frequentemente excede a capacidade da memória de trabalho, razão pela qual utilizamos auxílios externos, como pedaços de papel e softwares de depuração.

Os novatos precisam de rastreamento concreto com frequência, mas não têm experiência em selecionar as “partes móveis” certas para acompanhar na representação do estado, fazendo com que eles falhem como resultado de sobrecarga cognitiva excessiva [Vainio e Sajaniemi 2007; Fitzgerald et al. 2008].

Um exemplo de mecanismo de enfrentamento de iniciantes pode ser o “rastreamento de valor único” observado por Vainio e Sajaniemi, que limita o número de valores de variáveis não triviais na representação do estado mental a um único valor: apenas um “espaço” sem nome é usado para armazenar o valor da variável que recebeu o valor mais recentemente e que não é diretamente visível no código. O rastreamento de valor único se baseia em um modelo mental incorreto da máquina conceitual, mas o iniciante pode nem perceber isso, já que a abordagem “funciona bem com pequenos programas típicos de cursos básicos de programação”.

Robustez. Uma teoria de modelos mentais foi proposta por de Kleer e Brown [1981; 1983], que argumentam que apenas certos tipos de modelos mentais, que atendem a certos “princípios estéticos”, se prestam a responder a perguntas inesperadas sobre sistemas. De Kleer e Brown afirmaram que o uso de tais modelos mentais de alta qualidade, que os autores denominaram *robustos*, é característico do comportamento de especialistas. A robustez surge de submodelos que representam componentes do sistema: os componentes de um modelo robusto são compreendidos em termos de conhecimento geral pertinente a esses componentes, em vez de conhecimento específico pertinente à configuração particular dos componentes. Um modelo não robusto pode ser útil no contexto de um sistema específico em circunstâncias normais. Um modelo robusto é necessário para a transferência para um problema semelhante, porém novo, e para lidar com situações excepcionais, como o mau funcionamento de componentes.

Isso torna os modelos robustos altamente desejáveis.

Se a teoria de de Kleer e Brown for válida, e assumindo que os programas de computador são mecanismos causais do tipo ao qual sua teoria se aplica, então, para que os alunos transfiram sua compreensão de um programa para outros, eles precisam de um modelo robusto do programa original. Um modelo mental robusto também seria necessário para a depuração. Os componentes de um programa são as construções de programação usadas no código do programa.

Em um modelo robusto, esses componentes do programa são compreendidos em termos de sua semântica geral, ou seja, em termos do que eles significam para a máquina conceitual.

Vainio, que aplicou a teoria de Kleer e Brown à programação [Vainio e Sajaniemi 2007; Vainio 2006], discute o seguinte fragmento de código:

```
para (i = 0; i < 10; i++)
    { System.out.println(i);
    }
```

Uma descrição razoável deste laço `for` é: “Primeiro, a variável — `i` neste caso — é definida como zero. Em seguida, o laço itera sobre todos os valores de `i` de 0 a 9 e os imprime.” Um programador iniciante pode formar um modelo mental do código que corresponda a essa descrição e seja viável para lidar com esse programa específico, mas não é robusto. Por exemplo, alguns dos alunos no estudo de Vainio pensaram que *qualquer* variável usada dentro do corpo de um laço `for` é sempre definida como zero no início do laço. Tal entendimento claramente não é viável de forma geral. Também não é robusto: a semântica de uma instrução `for` (um componente do programa) é confundida com a finalidade da instrução `for` no contexto específico. Vainio e Sajaniemi [2007] descrevem tais violações dos princípios de robustez como comuns, atribuindo isso em parte à tendência, em cursos de Introdução à Ciência da Computação, de associar cada tipo de problema a apenas um único.

tipo de construção de programação, e cada construção de programação com um único tipo de problema.

Resumo da Seção. A teoria dos modelos mentais contribuiu para a nossa compreensão dos tipos de representações mentais intuitivas que programadores iniciantes formam de máquinas conceituais, frequentemente baseadas inicialmente em características superficiais da linguagem e analogias. Além disso, a teoria sugere que é necessária uma experiência substancial com uma determinada máquina conceitual antes que a transferência para máquinas superficialmente diferentes tenha probabilidade de sucesso. A habilidade fundamental de rastreamento de programas pode ser vista como a "execução" de um modelo mental que engloba tanto o programa quanto a máquina. Dois desafios para a execução bem-sucedida de um modelo mental são manter o controle do estado do programa na memória de trabalho e a dificuldade de formar modelos mentais que sejam robustamente fundamentados na semântica de tempo de execução livre de contexto de cada construção.

5. MÁQUINAS CONCEITUAIS E CONSTRUTIVISMO

O princípio central do paradigma educacional conhecido como construtivismo é que as pessoas constroem ativamente o conhecimento, em vez de recebê-lo e armazená-lo passivamente. O conhecimento não é assimilado de um mundo externo, nem é uma cópia do que um livro didático ou um professor disse. Em vez disso, o conhecimento é único para a pessoa ou o grupo que o construiu. A aprendizagem ocorre à medida que o conhecimento prévio e os interesses dos aprendizes interagem com novas experiências. Com base nisso, os construtivistas geralmente promovem pedagogias nas quais os aprendizes têm papéis ativos.

A Seção 5.1 apresenta uma visão geral do construtivismo (baseada principalmente em Phillips [1995], Steffe e Gale [1995], Laroche et al. [1998], Phillips [2000] e Tobias e Duffy [2009]). Na Seção 5.2, apresento e comento o que tem sido dito na literatura de CER sobre a relação entre construtivismo e a máquina conceitual.

5.1. Diferentes Construtivismos

É praticamente impossível encontrar um pesquisador da área da educação na atualidade que acredite que a aprendizagem se resume à transmissão, ou "despejo", de conhecimento preexistente de um professor ou livro para os alunos. Autodenominar-se construtivista é politicamente correto; negar o papel ativo dos aprendizes na construção do conhecimento é atrair desprezo. "Há um sentido muito amplo e vago em que todos nós hoje em dia somos construtivistas" [Phillips 1995]. No entanto, variamos em quão construtivistas somos e em como exercemos o construtivismo.

Diferentes vertentes do construtivismo. Os construtivistas divergem entre si quanto à perspectiva de se considerarem as mentes individuais ou os grupos sociais (ou ambos) como agentes construtores do conhecimento. *Os construtivistas pessoais* enfatizam a construção idiossincrática do conhecimento pelos indivíduos. *Os construtivistas sociais*, por sua vez, enfatizam a importância da natureza social e cultural da construção do conhecimento pelos indivíduos e tendem a ver o conhecimento como algo que se define pela colaboração social e pelo uso da linguagem.

Alguns construtivistas se contentam com uma noção bastante abstrata de construção do conhecimento; outros recorrem a diversas teorias para explicar os detalhes. Por exemplo, os construtivistas pessoais se baseiam em teorias como a dos modelos mentais, a dos esquemas e a da mudança conceitual, entre outras.

Existem também muitos "construtivistas pedagógicos" que não tomam posição sobre questões teóricas e se concentram no uso de pedagogias construtivistas, como a aprendizagem baseada em problemas ou a aprendizagem baseada em investigação.

Graus de Construtivismo. Os puristas de várias tradições construtivistas adotam posições epistemológicas extremas. Eles argumentam que o que chamamos de conhecimento deve ser discutido independentemente da ontologia, como um mundo real — se é que existe — ainda que incognoscível. De acordo com essas formas extremas de construtivismo, o que sabemos é apenas o nosso próprio conhecimento (ou o conhecimento social).

A construção do conhecimento (do grupo) não pode ser considerada verdadeira ou falsa em relação a uma realidade externa. O meio-termo entre os puristas e os realistas tradicionais é ocupado pelos moderados, que concordam com os construtivistas que o conhecimento não está firmemente fundamentado em uma realidade externa e que não podemos ter certeza do valor de verdade das afirmações de conhecimento. No entanto, os construtivistas moderados argumentam que a realidade impacta o conhecimento que as pessoas constroem e afeta a utilidade desse conhecimento e, portanto, tem um lugar na epistemologia [Phillips 1995; 2000].

5.2. Construtivismos no debate sobre a máquina conceitual

Embora formas de construtivismo tenham impactado o ensino da computação por décadas, foi apenas recentemente que o construtivismo se tornou uma força claramente visível na literatura de Pesquisa Computacional Estratégica (CER) [Greening e Kay 2001]. O construtivismo, de uma forma ou de outra, é cada vez mais mencionado por educadores da computação — seja como um termo da moda ou como uma estrutura teórica genuína — à medida que buscam justificar e fundamentar intervenções pedagógicas no ensino de programação, motivar questões ou abordagens de pesquisa, ou como uma ferramenta analítica que justifica retroativamente a pedagogia existente. Nesta revisão, foco-me no que foi dito sobre a máquina conceitual em termos construtivistas; a discussão em duas publicações (Greening [1999] e Ben-Ari [2001]) constitui a espinha dorsal desta parte da revisão.

Assim como o construtivismo em geral está longe de ser uma posição única e bem definida, os estudiosos dentro da Pesquisa Comparativa de Eficácia (CER) divergem bastante em como percebem as implicações das (variantes do) construtivismo para o ensino introdutório de programação.

Greening: Múltiplas Perspectivas. Greening [1999] defende uma forma de construtivismo forte na pedagogia. Ele argumenta que o principal desafio da aprendizagem de programação, do ponto de vista construtivista, não é a aquisição de conhecimento sobre linguagens de programação, sintaxe e semântica. Em vez disso, os aprendizes devem passar a ver a programação como uma atividade essencialmente criativa que envolve as habilidades de síntese e resolução de problemas. A verdadeira habilidade em programação envolve a capacidade de lidar com problemas complexos e mal estruturados em contextos autênticos. Greening afirma ainda que um construtivismo forte é muito necessário na educação moderna em ciência da computação, visto que alunos e professores são forçados a lidar com a explosão de informações e com as “múltiplas visões de mundo em constante mudança”. Greening promove a *aprendizagem baseada em problemas* [Savery e Duffy 1995] como um exemplo de pedagogia adequada para atingir esses objetivos.

Quando se trata de vários aspectos humanos da computação, como práticas de engenharia de software, bom design e usabilidade, Greening está sem dúvida certo ao enfatizar a utilidade de aprender a lidar com múltiplos pontos de vista. No entanto, quando se trata de aprender a lidar com o próprio computador, uma linha de pensamento diferente, inspirada pelo construtivismo, é sugerida por Ben-Ari [2001]. O computador não negocia...

Ben-Ari: Conhecimento Prévio do Computador. Ben-Ari [2001] discute a aplicação do construtivismo ao ensino da computação, focando principalmente no que poderia ser caracterizado como um construtivismo pessoal moderado de inclinação cognitivista. Ele enfatiza como o construtivismo destaca a importância do conhecimento prévio dos aprendizes para a aprendizagem e extrai uma série de conclusões que “parecem decorrer diretamente dos princípios construtivistas”. A tese de Ben-Ari se baseia em quatro afirmações.

- O conhecimento de informática é um pré-requisito para a compreensão da computação como a conhecemos. Sei disso.
- Os iniciantes em computação carecem de modelos eficazes de computadores.
- Os aprendizes necessariamente constroem conhecimento sobre os fenômenos que encontram, pois Melhor ou pior.
- O computador forma uma “realidade ontológica acessível”.

Ben-Ari argumenta que, de acordo com os princípios construtivistas (cognitivos), quando os aprendizes se deparam com um sistema, eles constroem um modelo mental dele. A construção do conhecimento é inevitável e ocorrerá independentemente de o aprendiz ter recebido um modelo conceitual normativo do fenômeno, embora a instrução possa influenciar o modelo resultante [Ben-Ari 2001; Ben-Ari e Yeshno 2006]. Ben-Ari reúne evidências da literatura (ver Seções 3 e 4) para sustentar seu argumento de que, ao se depararem com abstrações como código de programa, os alunos necessariamente construirão seu próprio conhecimento da máquina conceitual. Infelizmente, “modelos intuitivos de computadores estão fadados à inviabilidade”.

Ao enfatizar a “realidade ontológica acessível”, Ben-Ari argumenta que a computação (pelo menos as partes que envolvem diretamente computadores) possui uma ontologia que se reflete em conhecimento útil; isso contraria a rejeição construtivista radical da ontologia como base epistemológica. O computador é um artefato que se comporta de uma determinada maneira e, a menos que a compreensão do computador pelo aprendiz seja suficientemente próxima dessa realidade, logo haverá consequências. O feedback sobre compreensões inviáveis costuma ser imediato e brutal, na forma de mensagens de erro, travamentos e bugs. Além disso, embora as especificidades da compreensão que as pessoas constroem sobre o computador sejam únicas, todas as compreensões viáveis devem corresponder ao modelo normativo de forma bastante precisa, deixando pouco espaço para discordâncias. Como Ben-Ari [2001] aponta, “não faz muito sentido negociar modelos de sintaxe ou semântica de uma linguagem de programação” uma vez tomada a decisão de usar uma linguagem de programação específica. E assim se vai o “espectro de visões” [Greening 1999] que os educadores construtivistas geralmente esperam que os alunos explorem!

O Papel da Máquina. Greening se mostra cético quanto à afirmação de que um modelo do computador é necessário para o aprendizado da computação e observa que o construtivismo (forte) se preocupa menos em prescrever qual conhecimento prévio é necessário do que em reconhecer que o conhecimento anterior desempenha um papel na construção do conhecimento. Ele argumenta ainda que um modelo do computador será menos importante em um futuro (próximo?).

“Cada vez mais, talvez como um sinal de maturidade da(s) disciplina(s) da computação, a necessidade de compreender a máquina irá dissipar-se. Esta afirmação certamente horrorizará alguns leitores” [Greening 1999].

Greening parece estar falando sobre a importância de compreender a máquina em si num nível relativamente baixo e, nesse aspecto, tem um ponto válido. Os níveis de abstração na programação parecem estar aumentando, o que pode de fato significar que a máquina de baixo nível está se tornando cada vez menos importante para o ensino introdutório de programação. No entanto, a menos que a computação e a natureza da execução de programas mudem drasticamente, continua sendo fácil concordar com o argumento de Ben-Ari [2001] de que os alunos precisam de uma compreensão “um nível abaixo” do nível principal visado (do código do programa) que seja viável para explicar os fenômenos nesse nível. O nível de abstração da máquina conceitual necessário pode aumentar no futuro, juntamente com o nível de abstração das linguagens de programação, mas uma máquina conceitual ainda será necessária para os futuros programadores.

Resumo da Seção. Uma forma moderada de construtivismo, que enfatiza a necessidade de identificar o conhecimento prévio, pode ser interpretada como um apoio à afirmação de que é crucial para programadores iniciantes desenvolverem uma compreensão do computador que corresponda a um modelo normativo. Esse modelo não precisa ser o computador em si, mas uma máquina conceitual que opera logo abaixo das abstrações nas quais os aprendizes se concentram principalmente. Formas mais fortes de construtivismo, por outro lado, questionam normas educacionais que dependem da ontologia; os defensores dessas visões podem considerar o aprendizado sobre o computador como menos importante.

6. MÁQUINAS CONCEITUAIS E FENOMENOLOGRAFIA

A *fenomenografia* é uma tradição primordialmente qualitativa de pesquisa empírica, que evoluiu de um método de pesquisa utilizado na década de 1970 para uma abordagem teoricamente fundamentada e metodologicamente variada da pesquisa empírica sobre a experiência humana. A Seção 6.1 apresenta inicialmente alguns fundamentos da pesquisa fenomenográfica (principalmente com base em Marton e Booth [1997], Marton [2000] e Pang [2003]), que a Seção 6.2 relaciona ao conceito de máquina.

6.1. Fenomenografia Os

fenomenógrafos exploram a variação, de importância educacional, na forma como as pessoas vivenciam, percebem ou compreendem diversos fenômenos. Um fenômeno de interesse pode ser específico, como números na matemática do jardim de infância, matéria na física ou variáveis na computação, ou mais genérico, como aprender a programar ou mesmo aprender em geral.

O conceito central da abordagem fenomenográfica é o de uma forma de vivenciar um fenômeno.

Formas de Experienciar. As pessoas vivenciam o mesmo fenômeno de maneiras diferentes. Até mesmo a mesma pessoa vivencia o mesmo fenômeno de forma diferente em momentos e contextos diferentes. Existem inúmeras maneiras significativamente diferentes de vivenciar o mesmo fenômeno? Os fenomenógrafos postulam que não, e que um pesquisador pode descrever as formas de vivenciar um fenômeno de maneira abstrata, capturando as diferenças reveladoras entre as poucas maneiras qualitativamente distintas de vivenciá-lo.

Um fenômeno, visto de uma perspectiva particular, é caracterizado por um número limitado de *aspectos críticos* que o distinguem de outros fenômenos. Cada forma de compreender um fenômeno é definida pelos aspectos críticos que são discernidos; os contrastes entre o que é discernido e o que não é definem as diferenças qualitativas entre as formas de vivenciá-lo.

Para dar um exemplo da programação, um estudo descobriu três maneiras diferentes pelas quais alunos de Ciência da Computação do primeiro ano (CS1) entendem o que é um objeto [Eckerdal e Thune 2005]. A primeira categoria é muito simples: um objeto é um trecho de código. Aqui, apenas o aspecto crítico do texto do programa está em foco. Em uma compreensão mais rica, os objetos são vistos adicionalmente como entidades ativas durante a execução de um programa; essa relação entre objetos e eventos em tempo de execução é outro aspecto crítico dos objetos. Na terceira categoria, um aspecto de modelagem do mundo também é atribuído aos objetos. Os resultados deste estudo são típicos da fenomenografia, visto que as categorias formam uma hierarquia lógica na qual algumas compreensões são mais ricas, ou seja, aspectos críticos adicionais são discernidos nelas.

Aprendizagem como mudança na experiência. A forma mais importante de aprendizagem, do ponto de vista fenomenográfico, envolve tornar-se capaz de discernir aspectos críticos de um fenômeno, de modo que o fenômeno como um todo seja vivenciado de uma maneira nova e qualitativamente diferente.

Os fenomenógrafos enfatizam que os desafios da aprendizagem estão intrinsecamente ligados ao conteúdo que está sendo aprendido e aos objetivos específicos de aprendizagem. Eles desvalorizam ou até mesmo questionam as próprias ideias de representações mentais genéricas (por exemplo, modelos mentais), processos genéricos de aprendizagem e recursos pedagógicos genéricos. Em vez disso, recomenda-se que pesquisadores e professores se concentrem nas relações entre os aprendizes e os fenômenos que aprendem, e considerem a pedagogia em termos dos aspectos críticos específicos que se espera que os aprendizes aprendam a discernir.

6.2. Vivenciando a Programação Introdutória Parte do

trabalho fenomenográfico dentro do CER relaciona-se com a dinâmica de programas e a máquina conceitual.

Tabela I. Diferentes maneiras de vivenciar a programação

Descrição do código	
UM	A programação de computadores é vivenciada como o uso de uma linguagem de programação para escrever programas. textos.
B	A programação de computadores é vivenciada como descrito acima, e como uma forma de pensar que se relaciona com... Instruções na linguagem de programação que indicam o que acontecerá quando o programa for executado.
C	A programação de computadores é vivenciada como descrito acima, e também como a produção de aplicativos do tipo familiar do dia a dia.
D	A programação de computadores é vivenciada como descrito acima, e como um "método" de raciocínio que possibilita problemas a serem resolvidos.
E	A programação de computadores é vivenciada conforme descrito acima e é uma habilidade que pode ser usada fora do contexto profissional. curso de programação e para outros fins que não sejam programação de computadores.

Adaptado de Thune e Eckerdal [2010].

Vivenciando Conceitos Específicos. Alguns estudos investigaram as maneiras pelas quais Estudantes de programação vivenciam conceitos e estruturas específicas. Um tema emergente. Um dos resultados desses estudos é que programadores iniciantes às vezes se concentram exclusivamente em... sobre o aspecto estático "óbvio" dos conceitos que é visível no texto do programa. Eckerdal e O estudo de Thune sobre concepções de objetos [Eckerdal e Thune 2005], citado acima, é um exemplo desse tipo de trabalho; os autores também relatam uma categorização análoga de entendimentos do conceito de classe. Um estudo anterior sobre maneiras de vivenciar a recursão descobriu, de forma semelhante, que a recursão às vezes é percebida apenas como uma construção de código. em vez de em termos de comportamento em tempo de execução [Booth 1992].

Vivenciando a programação em geral. Ainda mais interessante para os propósitos atuais. são os estudos fenomenográficos nos quais o fenômeno de interesse foi mais amplo. Desenvolvendo-se a partir do trabalho pioneiro de Booth, vários pesquisadores estudaram como os programadores novatos percebem os fenômenos da *programação* e *da aprendizagem de programação* [Booth 1992; Bruce et al. 2004; Stoodley et al. 2004; Eckerdal et al. 2005; Thune e Eckerdal 2009; 2010]. Este conjunto de trabalhos aponta amplamente na mesma direção geral, com estudos que apoiam as descobertas de Booth e elaboram alguns temas. Como um Um exemplo representativo é a Tabela I, que mostra a categorização de Thune e Eckerdal sobre as formas como os alunos entendem o que é programação. Assim como nos estudos de programação específica. conceitos, a primeira categoria descreve uma forma relativamente pobre de compreensão em que O comportamento em tempo de execução é ignorado.

Um programador iniciante que encara o aprendizado de programação simplesmente como aprender a... Escrever o texto do programa seguindo regras sintáticas inevitavelmente resultará na perda de muitas funcionalidades. oportunidades de aprendizado até que sua visão geral da programação mude significativamente. Resultados fenomenográficos destacaram os aspectos críticos da programação. que constituem desafios para o ensino de programação. Um desses desafios é o aspecto de tempo de execução que é discernido na Categoria B da Tabela I, mas não na Categoria A—corresponde aproximadamente à máquina conceitual; na Categoria A, a programação não é percebido como escrever instruções para a máquina executar.

Resumo da seção. Programadores iniciantes às vezes não conseguem perceber a programação. Construções como algo mais do que código e programação como algo mais do que a produção de código. Uma das coisas que falta nessas concepções é a ação que ocorre. quando os programas são executados. O conhecimento limitado restringe a capacidade de programação e oportunidades para aprendizado adicional. O trabalho fenomenográfico dentro da CER pode ser interpretado como um apoio à necessidade de iniciantes aprenderem sobre uma máquina conceitual que Relaciona o texto do programa com a atividade em tempo de execução dentro do computador.

7. MÁQUINAS CONCEITUAIS E TEORIA DO CONCEITO DE LIMIAR

Meyer e Land [2003; 2006] propõem que, embutidas nas disciplinas acadêmicas, existem barreiras problemáticas à compreensão do aluno, que eles denominam *conceitos limiares*.

Essas “joias do currículo” representam pontos transformadores nas experiências de aprendizagem dos alunos, permitindo-lhes enxergar outros conceitos sob uma nova perspectiva. Os conceitos-chave propostos em programação incluem dinâmica de programas, ocultação de informações e interação entre objetos.

Os conceitos limiares (CLs) ainda são uma estrutura teórica relativamente recente que requer maior suporte empírico. No entanto, eles são obviamente uma base fértil para discussão e explorações pedagógicas, como evidenciado pelo rápido crescimento da literatura sobre CLs nos últimos anos. O trabalho em andamento sobre conceitos limiares pode nos ajudar a obter mais informações sobre por que alguns alunos parecem não aprender “nada do conteúdo”, enquanto outros parecem absorver “tudo” — um fenômeno comum em cursos introdutórios de Ciência da Computação. A resposta pode estar no próprio currículo: a teoria dos CLs sugere que certos conteúdos particularmente transformadores e integradores deixam muitos alunos estagnados e incapazes de progredir até que consigam enxergar as conexões entre conceitos relacionados.

7.1. Características dos Conceitos Limiares Um

conceito limiar não é um mero “conceito central”.³ Para se qualificar como um CT, um conceito deve atender a uma definição mais rigorosa, embora ainda esteja sendo debatida. As características propostas para os conceitos limiares incluem o seguinte [Meyer e Land 2006; Land e Meyer 2008; ver também Sorva 2010].

- Um conceito limiar *transforma* significativamente a forma como o aluno percebe um assunto ou parte disso, e talvez até mesmo ocasionar uma mudança na identidade pessoal.
- Um conceito limiar *integra* amplamente outros conteúdos, expondo sua inter-relação.
- Um conceito fundamental *leva a uma nova forma de pensar e praticar* dentro de uma área temática; o aluno torna-se capaz de perceber conceitos, problemas, soluções e a própria disciplina como um membro pleno da comunidade acadêmica.
- Tal transformação é provavelmente *irreversível*, ou seja, improvável que seja esquecida ou desfeito.
- Um conceito limiar pode ser *muito problemático* para os alunos por uma série de razões, incluindo complexidade conceitual, taciturnidade na prática especializada, aparente falta de significado e caráter contraintuitivo.
- A aprendizagem de um conceito limiar geralmente envolve um *estado liminar potencialmente longo*, durante o qual o aprendiz oscila entre “saber” e “não saber”, pode tentar imitar um comportamento presumivelmente correto e frequentemente experimenta fortes emoções negativas.
- Os conceitos limiares tendem a envolver a transformação de uma ou mais *ideias “óbvias” do dia a dia* em formas específicas de cada disciplina (por exemplo, a ideia geral de abstração em ocultação de informação).
- Um conceito limiar pode *marcar limites* no “espaço conceitual” entre disciplinas ou escolas de pensamento.

7.2. Dinâmica de Programas como Conceito Limiar Uma

distinção crucial em programação é aquela entre a existência de um programa como código e sua existência como uma entidade dinâmica em tempo de execução dentro de um computador. O código é tangível e sua existência é fácil de perceber. O último aspecto de um programa—

³Na verdade, os conceitos-limiais podem não ser conceitos individuais com nomes bem estabelecidos, mas algo mais amplo; essa é uma questão que está sendo debatida pela comunidade de pesquisa em conceitos-limiais. Meu uso da palavra “conceito” em “conceito-limiar” pode ser interpretado como uma abreviação para “forma de compreender determinado conteúdo curricular”.

o programa executado por uma máquina hipotética é muito menos assim. Foi apresentado um argumento de que a dinâmica do programa constitui um importante limiar transformador que os programadores iniciantes devem ultrapassar. Reafirmo este argumento no texto seguinte (em grande parte reproduzido de Sorva [2010], ver também Vagianou [2006], Shinnars-Kennedy [2008] e Zander et al. [2008]).

Integração. Uma visão dinâmica de um programa reúne o código do programa, o estado do programa e o processo que o altera, bem como o computador no qual o programa é executado (se não o hardware real, pelo menos uma máquina hipotética). A capacidade de visualizar programas como dinâmicos é necessária para compreender genuinamente uma série de outros conceitos e distinções: variáveis e valores; declarações de função versus chamadas de função; classes, objetos e instanciação; expressões e avaliação; declarações de tipo estático versus tipos de tempo de execução; escopo versus tempo de vida; e assim por diante. O uso dinâmico da memória para acompanhar o estado do programa é fundamental para grande parte desse poder integrativo.

Transformação. Como ilustrado pelo trabalho fenomenográfico analisado na Seção 6, aprender a programar envolve uma mudança qualitativa na forma como os aprendizes passam a perceber os conceitos de programação e a própria programação não apenas em termos de código, mas em relação à máquina conceitual.

A dinâmica de programas transforma a noção universal e cotidiana de estado — que poderia ser chamada de “ideia fundamental” da computação [Schwill 1994; Sorva 2010] — em algo crucial para o modo como o programador pensa. Uma visão dinâmica de programas leva ao que Perkins [2006] chama de uma nova *episteme*, uma nova maneira de raciocinar sobre programas que é impossível a menos que o aluno tenha incorporado a noção de dinâmica de programas em seus pensamentos e práticas. Em particular, pensar em termos de dinâmica de programas torna possível a habilidade fundamental de rastreamento de programas (ver Seção 4.3).

Problemas. A dinâmica de programas é problemática. Praticamente qualquer estudante de programação pode afirmar, da boca para fora, que os programas são executados passo a passo, fazendo com que as coisas aconteçam dentro do computador. No entanto, nem todos esses estudantes internalizam genuinamente essa noção e a aplicam na prática. As seções anteriores forneceram inúmeras referências a estudos que demonstram as dificuldades que os iniciantes enfrentam para entender o que acontece quando um programa é executado.

Parte da dificuldade da dinâmica de programas reside em seu caráter tácito. Como observado na Seção 4.3, os programadores raramente explicitam a natureza dual dos programas, que lhes é óbvia — quando falamos de um “programa”, nos referimos ao código, ao que o código faz ao ser executado, ou a ambos simultaneamente. A centralidade da noção, antes considerada não problemática, de estado para a dinâmica de programas também pode ser contraintuitiva para iniciantes [Shinnars-Kennedy 2008].

Limites. Usuários finais e programadores têm perspectivas diferentes em relação aos programas de computador. Somente o último grupo sente-se diretamente envolvido no que acontece quando um programa é executado por um computador. Essa é uma das maneiras pelas quais a dinâmica do programa serve como um marcador de fronteira, separando programadores de não programadores. Um estudante que não analisa programas ou não reflete sobre a dinâmica de sua execução não está realmente pensando e praticando como um programador.

A dinâmica de programas também demarca duas escolas de pensamento dentro da computação: situa-se na fronteira entre a programação de computadores e a programação como matemática. Esta última *episteme*, que Dijkstra defendeu de forma famosa e controversa como a perspectiva de escolha para os cursos de Introdução à Ciência da Computação, é regida pela lógica formal e pelas provas, e a primeira pelos testes, pelo rastreamento mental e pelo raciocínio operacional [Dijkstra et al. 1989].

Irreversibilidade. Há poucas evidências baseadas em pesquisa sobre a irreversibilidade do aprendizado sobre a dinâmica de programas. No entanto, pelo menos o presente autor nunca ouviu falar de algo assim.

É raro um programador esquecer como enxergar programas como entidades dinâmicas e rastreáveis depois de internalizar esse conceito, e ele também não espera ouvir falar de alguém que se esqueça disso. Um sinal de irreversibilidade pode ser a dificuldade que alguns programadores experientes têm em perceber como a programação se apresenta ao iniciante que ainda não ultrapassou esse limiar inicial de "óbvio".

Resumo da Seção. A dinâmica de programas — o domínio da máquina conceitual — é um conceito limiar plausível na programação de computadores, pois parece possuir as principais características de tais conceitos. É integradora, transformadora, problemática para muitos aprendizes, situa-se na fronteira disciplinar e abre caminho para uma forma de raciocinar sobre programas como os programadores o fazem. Não conseguir ultrapassar esse limiar é um sério obstáculo para o aprendizado posterior.

8. IMPLICAÇÕES PEDAGÓGICAS

Nesta seção, considero as implicações pedagógicas do que foi dito, baseando-me de forma eclética nas teorias discutidas nas cinco seções anteriores e em suas respectivas recomendações pedagógicas.

8.1. A Máquina Conceitual como Objetivo de Aprendizagem:

Transformando as Perspectivas dos Alunos. Tanto a fenomenografia quanto a teoria dos conceitos limiares enfatizam que existem diferentes maneiras de perceber o conteúdo curricular. Consequentemente, os professores devem ter como objetivo a transformação das perspectivas dos alunos em relação ao conteúdo-chave. A menos que os alunos tenham acesso à perspectiva apropriada, o ensino subsequente será ineficiente, senão totalmente infrutífero. A estrutura dos conceitos limiares sugere uma abordagem de "menos é mais" para o ensino [Cousin 2006]: o professor deve concentrar seus esforços no conteúdo selecionado que transforma a visão do aluno sobre a disciplina e facilita a aprendizagem de outros conceitos, em vez de enterrar essas preciosidades conceituais em meio a uma vasta massa de conhecimento, onde podem passar praticamente despercebidas.

Recursão, parâmetros de referência e instanciação de objetos, por exemplo, são conceitos difíceis de compreender até mesmo para um iniciante capaz de pensar em programas em termos de seu aspecto dinâmico e acompanhar a execução passo a passo. Sem a "lente" da dinâmica do programa, dominar esses outros conceitos importantes torna-se praticamente impossível. Um professor de programação não deve cair na armadilha de presumir que os alunos pensam como ele. Em vez disso, os professores precisam ajudar os alunos a desvendar a natureza do "jogo subjacente" tácito. Nem o professor nem o aluno devem se contentar com a mera formalidade da ideia de que os programas são executados passo a passo e utilizam memória. Um aluno que conclui um curso de programação sem ter desenvolvido uma perspectiva dinâmica sobre a execução de programas não aprendeu realmente muito sobre programação de computadores, independentemente de quantas definições de conceitos tenha memorizado ou quantos modelos de código seja capaz de aplicar. Por outro lado, alguém que ultrapassou esse limiar está bem posicionado para aprender mais com relativa facilidade.

Ensinando sobre um Modelo de Máquina. Todo curso introdutório de programação envolve uma máquina conceitual; a máquina está implícita na linguagem de programação utilizada. No entanto, provavelmente é melhor ter o aprendizado sobre uma máquina conceitual como um objetivo explícito do que como um objetivo implícito.

De acordo com pesquisadores de modelos mentais, é útil usar um *modelo conceitual* — uma explicação elaborada com o propósito de elucidar a estrutura e o funcionamento interno de um sistema — no ensino. Um modelo conceitual pode ser uma simples metáfora ou analogia, ou uma explicação mais complexa do sistema. Os modelos conceituais têm se mostrado úteis em diversos contextos. Segundo pelo menos uma revisão, o estudo típico de modelos mentais conclui que os alunos aprendem "como funciona" utilizando um modelo conceitual.

demonstram melhor desempenho do que aqueles que receberam instruções de “como fazer” para controlar um sistema [Schumacher e Czerwinski 1992].

O uso de um modelo conceitual também é defendido por Ben-Ari em conexão com seu construtivismo moderado (ver Seção 5). Ben-Ari [2001] argumenta que, para evitar a construção malfadada de conhecimento intuitivo sobre o computador e a semântica da linguagem de programação, a instrução deve começar com o modelo subjacente antes de prosseguir para as abstrações baseadas nele. Isso está em consonância com as descobertas de pesquisadores de modelos mentais de que mudar um modelo inicial tende a ser mais difícil do que formá-lo. Outra preocupação é que um modelo seriamente falho de uma máquina conceitual pode funcionar para explicar o comportamento de alguns exemplos de programas, mesmo que falhe de forma geral, o que reforça ainda mais a crença do aluno em sua compreensão atual.

Embora os alunos obviamente não cheguem como folhas em branco, independentemente do que os professores façam, começar o mais cedo possível parece uma boa ideia. Um modelo conceitual pode explicitar as maneiras pelas quais o comportamento da máquina difere do pensamento humano e pode destacar como uma linguagem de programação difere da linguagem natural e da matemática familiar.

Além disso, um modelo conceitual de uma máquina hipotética pode servir como base para o ensino sobre rastreamento de programas. O modelo conceitual pode sugerir uma perspectiva adequada para os alunos adotarem ao rastrear programas. Também pode fornecer uma plataforma para professores e alunos discutirem o que deve ser acompanhado durante o rastreamento e o papel das representações externas.

A Alternativa: Uma Máquina Implícita. De forma alguma todos os professores de Introdução à Ciência da Computação ensinam explicitamente aos seus alunos sobre uma máquina conceitual. Nem todos concordam que isso faça sentido. Como discutido na Seção 5, formas fortes de construtivismo podem levar à rejeição da ontologia como base epistemológica e à rejeição da importância de ensinar explicitamente um modelo normativo do computador. Greening [1999] afirma que, com o emprego de práticas construtivistas, como a aprendizagem baseada em problemas, problemas sérios não surgirão porque os alunos descobrirão compreensões viáveis do computador à medida que trabalham em projetos grandes e complexos: “Um ambiente construtivista não encontraria alunos escrevendo os fragmentos de código triviais necessários para permitir que tais concepções errôneas escapem”. Se este for o caso, as implicações para o ensino introdutório de programação são significativas; no entanto, a literatura sobre CER (Evidências de Computação Relevante) atualmente parece fornecer poucas evidências concretas que sustentem esse argumento.

8.2. Técnicas de Ensino

Supondo que desejamos ensinar explicitamente sobre dinâmica de programas e uma máquina conceitual, o que devemos fazer na prática?

Pedagogia para Conceitos Limiares. A literatura sobre conceitos limiares apresenta diversas sugestões pedagógicas [Meyer e Land 2006; Land e Meyer 2008]. Por exemplo, os professores devem procurar informar os alunos sobre a existência de conceitos limiares e liminaridade, aumentar a metacognição dos alunos sobre seus estados liminares e ajudá-los a lidar com a incerteza e as questões emocionais envolvidas. Os alunos precisam se envolver na manipulação ativa e consciente de cada conceito limiar para internalizá-lo. Deve-se ajudar os alunos a tomar consciência das maneiras como pensam e agem atualmente e motivá-los a transformar essas maneiras. Os tipos de imitação motivados por uma tentativa genuína de cruzar um limiar devem ser vistos como positivos, e não negativos.

Para seguir esse conselho, os professores de CS1 devem tentar ajudar seus alunos a perceber a importância do conceito (candidato) de dinâmica de programas e o quão problemático ele pode ser para muitos estudantes. Devem encorajar os alunos a desenvolverem a autoconsciência sobre como pensam a respeito de programas e raciocinam sobre o que os programas fazem. Uma maneira de alcançar isso pode ser por meio de visualizações e metáforas que concretizem...

Os aspectos dinâmicos dos programas. Ao tornar a dinâmica dos programas tangível, o professor pode não apenas ajudar o aluno a pensar sobre os programas de forma dinâmica, mas também torná-lo mais consciente do que está fazendo. Um aluno que tem consciência metacognitiva de pensar na execução do programa como um processo dinâmico passo a passo pode achar mais fácil compreender os princípios gerais incorporados no conceito fundamental e relacioná-los a múltiplos contextos. A exposição às dificuldades de outros alunos com o mesmo problema pode ajudar os aprendizes a lidar com os desafios afetivos da aprendizagem profunda; técnicas de ensino entre pares e trabalho em grupo podem ser úteis nesse sentido.

Visualização de Programas. Um meio relativamente popular de tornar concreta uma máquina conceitual é visualizá-la e seu comportamento. Isso pode ser feito desenhando em papel ou quadro-negro, ou em software de apresentação; outra alternativa é usar software de visualização desenvolvido para visualizar a execução de programas. O exemplo mais conhecido desse tipo de software é o depurador visual, que fornece um rastreamento passo a passo de um programa de entrada à medida que ele é executado, explicitando o fluxo de controle, os valores das variáveis, os quadros na pilha de chamadas e assim por diante.

Os depuradores visuais comuns são projetados pensando no programador experiente. Educadores da área de computação desenvolveram diversos sistemas de software mais voltados para o aprendizado, com o objetivo de visualizar aspectos da máquina para programadores iniciantes. Alguns desses sistemas são destinados exclusivamente à visualização do comportamento em tempo de execução relacionado a um conceito específico, como ponteiros, passagem de parâmetros, objetos ou atribuição; outros sistemas são mais genéricos. Uma revisão de ferramentas genéricas para visualização da dinâmica de programas em CS1 pode ser encontrada em um artigo complementar [Sorva et al., a ser publicado].

O impacto das atividades de aprendizagem. Tarefas de aprendizagem que ativam os alunos são apoiadas por muitas teorias de aprendizagem, atualmente talvez mais proeminentemente por várias formas de construtivismo. O objetivo dessas tarefas é ativar cognitivamente os alunos; muitos construtivistas, por exemplo, argumentam que isso pode ser alcançado exigindo ações concretas por parte dos alunos, em vez de fazê-los ouvir uma palestra ou visualizar uma apresentação criada pelo professor.

O trabalho de Wickens e Kessel sobre o desenvolvimento de modelos mentais oferece uma perspectiva interessante sobre o impacto das atividades de aprendizagem no conhecimento que os aprendizes constroem sobre sistemas causais [Kessel e Wickens 1982; Schumacher e Czerwinski 1992]. Eles estudaram o desempenho de pessoas treinadas alternadamente como "monitores", que supervisionam um sistema tecnológico complexo, ou como "controladores", que controlam o sistema manualmente. Como era de se esperar, Wickens e Kessel descobriram que o treinamento em monitoramento de sistemas melhora as habilidades de monitoramento das pessoas, e o treinamento em controle de sistemas melhora as habilidades de controle. No entanto, e de forma significativa, eles também descobriram que os controladores conseguiam transferir suas habilidades para tarefas de monitoramento, enquanto o inverso não era verdadeiro para os monitores. Os controladores também se mostraram mais eficientes na detecção de falhas do sistema a partir de pistas sutis que escapavam à atenção dos monitores. Seus resultados demonstram que pessoas realizando tarefas semelhantes, porém diferentes, na mesma máquina desenvolvem diferentes tipos de conhecimento sobre ela. Em particular, uma tarefa mais passiva resultou em pior aprendizado. Parece provável que o programador iniciante que se limita a estudar visualizações do computador executando programas tenha menos probabilidade de se tornar um bom "maquinista teórico" do que aquele que interage ativamente com o ambiente de execução do programa.

A visualização de programas para programadores iniciantes, assim como a visualização de software no ensino de computação em geral, tem um histórico comprovado, mas não é uma panaceia. Na última década, o papel das atividades de aprendizagem emergiu como um tema central na literatura de Pesquisa Comparativa em Engenharia (CER) sobre visualização, particularmente na visualização de algoritmos, mas também no contexto da visualização de máquinas conceituais para iniciantes. Em especial, pesquisas sugerem que a forma como os aprendizes interagem com as visualizações de software é mais importante, do ponto de vista da aprendizagem, do que as próprias técnicas de visualização.

[Hundhausen et al. 2002; Naps et al. 2003; Sorva et al., no prelo]. Alguns sistemas de visualização recentes buscam, em especial, envolver os alunos na resolução de questões sobre programas ou no controle manual da execução por meio de uma visualização. Fazer com que os alunos desenhem visualizações no papel ou simulem uma máquina hipotética em sala de aula são pedagogias alternativas que podem ser úteis para que os alunos se envolvam com a dinâmica dos programas [Sorva et al., no prelo]. Mesmo quando essas atividades não apresentam explicitamente uma "máquina", elas podem ajudar os alunos a compreender a execução de programas dentro de uma máquina hipotética [Andrianoff e Levine 2002; Borstler e Schulte 2005].

Varição nos Aspectos Críticos. Na tradição fenomenográfica (Seção 6), o trabalho do professor é visto principalmente como o de ajudar os alunos a discernir os aspectos críticos e associá-los ao fenômeno. O professor deve criar um "espaço de aprendizagem" apropriado [Marton et al. 2004] — uma situação de aprendizagem na qual os aspectos críticos visados estejam presentes, proporcionando assim uma oportunidade para aprender sobre o fenômeno. Em muitos casos, porém, apenas explicitar os aspectos críticos não é suficiente. Os alunos devem ser colocados em situações nas quais sintam necessidade de uma nova perspectiva sobre o fenômeno que estão estudando. Marton e Booth [1997] escrevem sobre a construção de uma *estrutura de relevância* para uma situação de aprendizagem — definida como a experiência de uma pessoa sobre o que uma situação exige — e incentivam os professores a "criar situações de aprendizagem nas quais os alunos se deparem com novas abstrações, princípios, teorias e explicações por meio de eventos que criem um estado de suspense". Além disso, sugere-se que os fenômenos desafiadores podem ser melhor compreendidos concentrando-se primeiro em aspectos críticos individuais separadamente e depois "fundindo" os aspectos para discernir seus relacionamentos e obter uma sensação holística do fenômeno [Marton et al. 2004].

No contexto da dinâmica do programa, a pedagogia baseada na fenomenografia e a teoria da variação da aprendizagem associada estão sendo exploradas por Thune e Eckerdal [2009]. A abordagem deles consiste em apresentar aos alunos exemplos em que as alterações no texto do programa são mínimas, mas têm um impacto drástico no comportamento do programa durante a sua execução. Uma vez que o aspecto dinâmico de um programa tenha sido evidenciado dessa forma, os alunos exploram a relação entre o texto do programa e o comportamento resultante utilizando ferramentas de software (por exemplo, um depurador visual).

O Impacto dos Ambientes. Paradigmas e ambientes de programação podem influenciar o aprendizado sobre a dinâmica de programas. Alguns ambientes existentes, e talvez especialmente os ambientes de programação do futuro, confundem a linha divisória entre o tempo de desenvolvimento e o tempo de execução do programa — considere, por exemplo, o ambiente Smalltalk, a IDE BlueJ [Kolling 2008], o tutor DISCOVER [Ramadhan et al. 2001] e o trabalho recente de Victor [2012]. Tais ambientes trazem muitos benefícios interessantes tanto para programadores iniciantes quanto para experientes. Eles também podem apresentar alguns desafios pedagógicos. Por exemplo, ao estudarem alunos do ensino médio aprendendo programação orientada a objetos, Ragonis e Ben-Ari [2005a, 2005b] "perceberam sérias dificuldades de aprendizado sobre a dinâmica de programas", visto que "os alunos têm dificuldade em criar uma visão geral da execução de um programa que resolve um determinado problema". Eles sugerem que a modelagem orientada a objetos e as ferramentas pedagógicas que envolvem a manipulação direta de objetos durante a criação de um programa (como o BlueJ) podem agravar essa dificuldade.

O impacto da ordem curricular. A forma como os alunos aprendem sobre o conceito de máquina pode ser afetada de diversas maneiras pela ordem dos tópicos no ensino. Argumenta-se, tanto em bases teóricas [Ben-Ari 2001] quanto com base em evidências empíricas diretas [Mayer 1976, 1981], que seria mais vantajoso ajudar os alunos a formar um modelo mental viável do computador antes de aprenderem a programar em uma linguagem de alto nível. Adiar as tarefas de programação pode ajudar a prevenir o surgimento de concepções errôneas, uma vez que os alunos já possuem conhecimento prévio para fundamentar sua compreensão.

de construções de programação (ver Seção 5.2). No entanto, essa abordagem envolve uma grande desvantagem, pois os alunos precisam dedicar mais tempo aos conceitos fundamentais antes de começarem a trabalhar com código e projetos de programação motivadores.

Podemos obter outra perspectiva sobre a ordenação de tópicos a partir de um estudo experimental de Kessler e Anderson [1986], no qual um grupo de novatos aprendeu primeiro programação recursiva, seguida de programação iterativa, e outro grupo foi apresentado aos tópicos na ordem inversa; nenhum dos grupos recebeu instrução explícita sobre uma máquina conceitual (veja também o estudo similar de Wiedenbeck [1989]). Os alunos que começaram com iteração inicialmente formaram melhores modelos mentais de fluxo de controle, que posteriormente conseguiram transferir para o novo contexto da programação recursiva. Em contraste, os alunos que começaram com programação recursiva tenderam a um estilo de programação baseado em modelos, no qual tentaram combinar as características superficiais dos problemas com as características superficiais de exemplos de programas conhecidos. Consequentemente, o grupo que aprendeu primeiro com recursão conseguiu resolver certos tipos de problemas recursivos, mas não conseguiu transferir o que sabia para a programação iterativa, ficando "sobrecarregado pelas diferenças superficiais entre recursão e iteração". Uma análise de protocolo sugeriu que o grupo que aprendeu primeiro com recursão não construiu um modelo dos princípios implícitos de fluxo de controle subjacentes aos programas de exemplo que viram; Em outras palavras, eles não haviam compreendido a máquina conceitual necessária. Os resultados de Kessler e Anderson destacam uma armadilha na transição da recursão para a iteração, o que significa que ou se deve começar com a iteração ou, alternativamente, prestar atenção especial ao modelo de execução dos programas ao passar da recursão para a iteração. De maneira mais geral, esses resultados enfatizam a importância de os professores de Ciência da Computação I examinarem as maneiras sutis pelas quais a ordem dos tópicos pode impactar o aprendizado sobre a dinâmica de programas.

9. MÁQUINAS CONCEITUAIS E PROGRAMAÇÃO ORIENTADA A OBJETOS

Um debate contínuo entre os profissionais diz respeito a saber se a programação orientada a objetos é um paradigma apropriado para programadores iniciantes [Bruce 2004; Lister et al. 2006]. Desde a década de 1990, as abordagens orientadas a objetos tornaram-se comuns, mas as abordagens procedurais mais tradicionais também continuam populares. Nesta seção, reviso o que foi dito sobre máquinas nocionais no contexto da programação orientada a objetos em CS1.

Seguindo as recomendações das seções anteriores, um professor que utiliza POO em Introdução à Ciência da Computação deve apresentar um modelo conceitual de uma máquina conceitual que explique o comportamento dinâmico de programas orientados a objetos em um nível razoável de abstração. Mas como seria uma máquina conceitual orientada a objetos apropriada? Diferentes pontos de vista foram expressos na literatura.

Uma Máquina Imperativa Estendida. Alguns argumentam que uma máquina conceitual adequada para programação orientada a objetos é uma extensão de uma máquina conceitual procedural ou imperativa. Sajaniemi e Kuittinen [2008] comparam duas máquinas conceituais para programação orientada a objetos e programação procedural, ambas descrevendo a execução no mesmo nível de abstração. Eles argumentam que uma máquina conceitual para programas imperativos muito simples precisa apresentar (apenas) variáveis, dispositivos de E/S e um contador de programa. Ela pode ser expandida facilmente para uma máquina conceitual mais rica adicionando ponteiros, uma pilha de chamadas e mecanismos para passagem de parâmetros e retorno de valores, assim que os alunos cheguem a programas que envolvam ponteiros e funções. Em contraste, Sajaniemi e Kuittinen argumentam que qualquer máquina conceitual orientada a objetos deve ser mais complexa, apresentando objetos, referências a objetos, uma pilha de chamadas, mecanismos para passagem de parâmetros e retorno de valores, variáveis, dispositivos de E/S e um contador de programa. Segundo Sajaniemi e Kuittinen, "não só o tamanho da máquina conceitual necessária é muito maior do que no caso procedimental, como também a máquina conceitual inicial necessária para compreender

Os primeiros programas também são muito mais complicados." Eles se baseiam na literatura (ver Seção 3) para afirmar ainda que "a máquina nocal OO é ainda menos compreendida pelos alunos do que a máquina nocal imperativa".

Schulte e Bennedsen [2006] mencionam brevemente uma máquina conceitual orientada a objetos que "compreende aspectos imperativos tradicionais da linguagem de programação, bem como uma compreensão da interação entre objetos que ocorre durante o tempo de execução".

Essa formulação também sugere uma máquina conceitual orientada a objetos que é uma extensão de uma máquina imperativa, mas que também envolve aspectos de nível superior.

Uma Máquina OO de Alto Nível. A máquina nocal orientada a objetos de Sajaniemi e Kuitinen opera no mesmo nível de abstração que sua máquina nocal procedural, lidando com primitivas como variáveis e quadros de pilha. Em contraste, Bergin [2000] enfatiza a maneira dramaticamente diferente como a computação é pensada em OOP.

Um componente bastante típico de um curso introdutório de programação usando o paradigma procedural é uma discussão sobre a arquitetura da máquina de von Neumann. . . . Este modelo de máquina simples se encaixa bem com o paradigma procedural, mas menos bem com outras maneiras, mais abstratas, de se encarar a computação.

Existe uma relação simples entre o nível físico fornecido pela arquitetura de von Neumann e o nível virtual fornecido pela maioria das linguagens procedurais. Isso não ocorre com o paradigma funcional ou orientado a objetos. O paradigma funcional, obviamente, oculta completamente a arquitetura física subjacente. O paradigma orientado a objetos não a oculta, mas a inverte. Em vez de os dados serem movidos para a CPU para processamento, uma metáfora muito comum em POO é que a CPU se move dentro dos objetos.

Em outro lugar, Gries [2008] critica o uso de abstrações de baixo nível no ensino de POO.

Mas muitos livros de programação falham ao usar a abstração de forma apropriada, por exemplo, descrevendo variáveis e atribuições em termos de computadores... Introduzir conceitos de computação em termos de computador pode criar detalhes desnecessários e confusos, especialmente quando conceitos de OO são descritos em termos de um computador, com discussões sobre ponteiros para objetos na memória, heaps e outros termos relacionados à implementação.

Gries prefere descartar terminologia difícil (por exemplo, "referência", "ponteiro"), pois "com a abstração apropriada do computador, esses termos se tornam desnecessários". Ele sugere um modelo conceitual metafórico de alto nível para a execução de programas em termos de objetos e classes.

De maneira semelhante, Caspersen e seus colegas buscaram representar uma máquina conceitual orientada a objetos em um nível de abstração mais elevado [Caspersen 2007; Bennedsen e Schulte 2006; Henriksen 2007]. Eles argumentam que os programadores orientados a objetos precisam entender a execução de programas em termos de uma máquina conceitual que lida com estruturas de objetos que interagem, e vislumbram o uso de um modelo conceitual que abstrairia detalhes como a avaliação de expressões e se concentraria nas interações.

Caspersen [2007] descreve um sistema futuro no qual os alunos poderiam "brincar" com a visualização de um modelo de objeto, avançando e retrocedendo, e fazendo alterações no estado do programa à vontade.

Dois Máquinas. Berglund e Lister [2007, 2010] descobriram, por meio de análise fenomenográfica, que entre os participantes do debate sobre aprendizado a partir de objetos, esse aprendizado é vivenciado de maneiras diferentes: como uma extensão da programação imperativa ou como algo conceitualmente bastante distinto da programação imperativa. Essa divisão qualitativa (na minha interpretação) se reflete na literatura sobre programação orientada a objetos.

Máquinas nocionais. A máquina nocional OO de Sajaniemi e Kuittinen é um exemplo da primeira percepção, enquanto Bergin representa a outra linha de pensamento.

A maioria dos autores, talvez por uma questão de simplicidade, fala de uma única máquina nocional procedural/imperativa e de uma única máquina nocional orientada a objetos, mais complexa. Outra maneira de pensar sobre o assunto é que, embora uma única máquina nocional possa ser suficiente para entender programas procedurais, a programação orientada a objetos efetivamente requer (pelo menos) duas máquinas nocionais diferentes. Uma pode ser vista como uma extensão habilitada para objetos de uma máquina nocional procedural, à la Sajaniemi e Kuittinen, e a outra descreve a troca de mensagens entre objetos que interagem. As duas máquinas nocionais operam em diferentes níveis de abstração e oferecem duas perspectivas diferentes sobre a programação orientada a objetos. Isso é consistente com a ideia de que a programação orientada a objetos é simultaneamente uma extensão da programação imperativa e algo conceitualmente diferente dela. Alunos iniciantes em programação orientada a objetos precisam aprender sobre ambas as máquinas nocionais — e seu relacionamento — desde cedo. Isso torna um curso introdutório de Ciência da Computação (CS1) orientado a objetos mais desafiador de ensinar com sucesso — exige-se mais do professor para não exigir demais dos alunos. Quando se obtém sucesso, no entanto, realiza-se um feito notável.

mais.

10. CONCLUSÕES

Schulte e Bennedsen [2006] pesquisaram as opiniões de professores de programação sobre a importância relativa de vários tópicos de CS1 e fizeram uma observação interessante.

Eles relatam que, embora os professores de programação considerassem alguns tópicos específicos relacionados à máquina conceitual (por exemplo, referências) relativamente importantes em comparação com outros tópicos específicos, a máquina conceitual, de forma mais geral, foi vista como relativamente pouco importante em comparação com o aprendizado sobre notação e pragmática, entre outras coisas. O resultado de Schulte e Bennedsen pode ser afetado pela falta de consenso entre os professores sobre o que exatamente significa "máquina conceitual" — o termo foi usado e brevemente explicado na pesquisa —, mas provavelmente também reflete o status quo no ensino de Introdução à Ciência da Computação, no qual a visão geral de como os programas funcionam em tempo de execução não recebe a atenção que merece à luz da teoria da aprendizagem e das evidências empíricas.

Este artigo reuniu diversas perspectivas sobre o papel das máquinas nocionais no ensino introdutório de programação. Catálogos de concepções errôneas, teorias de modelos mentais e construtivismo, pesquisas sobre as experiências de programação dos aprendizes e a teoria dos conceitos limiares corroboram a ideia de que programadores iniciantes precisam aprender sobre uma ou mais máquinas nocionais. Algum tipo de máquina conceitual está presente em toda introdução à programação, seja ela explícita ou não: está implícita na linguagem de programação e no paradigma utilizado. As máquinas nocionais não são o único gargalo responsável pelas dificuldades dos alunos — outros desafios incluem a falta de padrões de solução comuns, a necessidade de precisão sintática e questões motivacionais, por exemplo —, mas representam uma das principais fontes de dificuldade, que os instrutores de programação devem abordar explicitamente ao planejar seu ensino.

⁴Há algumas evidências limitadas de estudos de compreensão de programas que sugerem que os novatos que foram ensinados a programação orientada a objetos têm mais dificuldade em formar os chamados "modelos de programa" (representações mentais de um texto de programa, as operações elementares que ele executa, fluxo de controle, etc.), enquanto que dentro do paradigma procedural os novatos lutam relativamente mais em formar "modelos de domínio" (envolvendo, entre outras coisas, os objetivos e subobjetivos de um programa) [Wiedenbeck e Ramalingam 1999; Wiedenbeck et al.

1999]. Uma conjectura desses estudos é que os novatos em programação orientada a objetos podem precisar de ajuda adicional com a máquina conceitual de nível inferior em comparação com os novatos em programação procedural.

REFERÊNCIAS

- ADELSON, B. E SOLOWAY, E. 1985. O papel da experiência de domínio no projeto de software. *IEEE Trans. Softw. Eng.* 11, 11, 1351–1360.
- ANDRIANOFF, SK E LEVINE, DB 2002. Interpretação de papéis em um mundo orientado a objetos. *SIGCSE Bull.* 34, 1, 121–125.
- BABBAGE, C. 1864. *Passagens da Vida de um Filósofo*. Longman Green.
- BAYMAN, P. E MAYER, RE 1983. Um diagnóstico das concepções errôneas de programadores iniciantes sobre instruções de programação BASIC. *Comm. ACM* 26, 9, 677–679.
- BEN-ARI, M. 2001. Construtivismo no ensino de ciência da computação. *J. Comput. Math. Sci. Teach.* 20, 1, 45–73.
- BEN-ARI, M. E YESHNO, T. 2006. Modelos conceituais de artefatos de software. *Interact. Comput.* 18, 6, 1336–1350.
- BENNEDSEN, J. E SCHULTE, C. 2006. Um modelo de competências para interação com objetos na programação introdutória. Em *Anais do 18º Workshop do Grupo de Interesse em Psicologia da Programação (PPIG'06)*. 215–229.
- BERGIN, J. 2000. Por que o paradigma procedural é o primeiro paradigma errado se o objetivo é a POO. <http://csis.pace.edu/ybergin/papers/Whynotproceduralfirst.html>.
- BERGLUND, A. E LISTER, R. 2007. Debatedo o debate OO: onde está o problema? In *Anais da 7ª Conferência do Mar Báltico sobre Pesquisa em Educação Computacional (Koli Calling'07)*. Sociedade Australiana de Computação, 171–174.
- BERGLUND, A. E LISTER, R. 2010. Programação introdutória e o triângulo didático. In *Anais da 12ª Conferência Australiana de Educação em Computação (ACE'10)*. Sociedade Australiana de Computação, 35–44.
- BESNARD, D., GREATHEAD, D., E BAXTER, G. 2004. Quando os modelos mentais falham: coocorrências em dinâmica, sistemas críticos. *Int. J. Hum. Comput. Stud.* 60, 1, 117–128.
- BHUIYAN, SH, GREER, JE e MCCALLA, GI 1990. Modelos mentais de recursão e sua utilização no consultor de programação SCENT. In *Anais da Conferência Internacional sobre Sistemas Computacionais Baseados em Conhecimento (KBCS'89)*. Springer, 135–144.
- BONAR, J. E SOLOWAY, E. 1985. Conhecimento prévio de programação: uma importante fonte de equívocos em iniciantes. programadores. *Hum. Comput. Interact.* 1, 2, 133–161.
- BOOTH, S. 1992. Aprendendo a programar: uma perspectiva fenomenográfica. Tese de doutorado, Universidade de Gotemburgo.
- BRUCE, C., BUCKINGHAM, L., HYND, J., MCMAHON, C., ROGGENKAMP, M., E STOODLEY, I. 2004. Maneiras de vivenciar o ato de aprender a programar: um estudo fenomenográfico de alunos de programação introdutória na universidade. *J. Inf. Technol. Educ.* 3, 143–160.
- BRUCE, KB 2004. Controvérsia sobre como ensinar CS 1: Uma discussão na lista de discussão de membros do SIGCSE. *SIGCSE Bull.* 36, 4, 29–34.
- BRUCE-LOCKHART, MP E NORVELL, TS 2007. Desenvolvimento interativo de modelos mentais de programação de computadores via Web. In *Anais da 37ª Conferência Anual Frontiers in Education (FIE'07)*. IEEE, S3H–3–S3H–8.
- BORSTLER, J. E SCHULTE, C. 2005. Ensinando modelagem orientada a objetos com cartões CRC e jogos de RPG. “ Em *Anais da 8ª Conferência Mundial IFIP sobre Computadores na Educação (WCCE'05)*. IFIP TC-3.
- CANAS, JJ, BAJO, MT e GONZALVO, P. 1994. Modelos mentais e programação de computadores. *Int. J. Hum. Comput. Stud.* 40, 5, 795–811.
- CASPERSEN, ME 2007. Ensinando iniciantes em programação. Tese de doutorado, Departamento de Ciência da Computação, Universidade de Aarhus.
- CHEN, C.-L., CHENG, S.-Y., e LIN, JM-C. 2012. Um estudo sobre concepções errôneas e concepções ausentes de programadores Java iniciantes. In *Anais da Conferência Internacional sobre Fronteiras na Educação: Ciência da Computação e Engenharia da Computação (FECS'12)*.
- CORNEY, M., LISTER, R., E TEAGUE, D. 2011. Raciocínio relacional inicial e o programador iniciante: a troca como o “Olá Mundo” do raciocínio relacional. In *Anais da 13ª Conferência Australiana de Educação em Computação (ACE'11)*. Sociedade Australiana de Computação, 95–104.
- COUSIN, G. 2006. Uma introdução aos conceitos limiares. *Planet* 17, 4–5.
- DE KLEER, J. E BROWN, JS 1981. Modelos mentais de mecanismos físicos e sua aquisição. Em *Cognitivo Habilidades e sua aquisição*, JR Anderson, Ed., Lawrence Erlbaum, 285–309.
- DE KLEER, J. E BROWN, JS 1983. Pressupostos e ambiguidades em modelos mentais mecanicistas. Em *Mental Modelos*, D. Gentner e AL Stevens, Eds., Lawrence Erlbaum, 155–190.
- DIJKSTRA, EW, ET AL. 1989. Um debate sobre o ensino de ciência da computação [em resposta a Sobre a Crueldade de Dijkstra] de Really Teaching Computing Science]. *Comm. ACM* 32, 12, 1397–1414.

- DOUKAKIS, D., GRIGORIADOU, M., e TSAGANOU, G. 2007. Compreendendo o conceito de variável de programação com analogias interativas animadas. In *Anais da 8ª Conferência Helênica Europeia de Pesquisa em Matemática Computacional e suas Aplicações (HERCMA'07)*.
- Pesquisa, F. 1997. Avaliando as consequências cognitivas da abordagem orientada a objetos: uma revisão de estudos empíricos. DETIENNE sobre design orientado a objetos por indivíduos e equipes. *Interact. Comput.* 9, 1, 47–72.
- DETIENNE, F. E SOLOWAY, E. 1990. Uma estrutura de controle derivada empiricamente para o processo de programação. compreensão. *Int. J. Man. Mach. Stud.* 33, 3, 323–342.
- DU BOULAY, B. 1986. Algumas dificuldades de aprendizagem de programação. *J. Educ. Comput. Res.* 2, 1, 57–73.
- DU BOULAY, B., O'SHEA, T., e MONK, J. 1981. A caixa preta dentro da caixa de vidro: apresentando a computação conceitos para novatos. *Int. J. Man. Mach. Stud.* 14, 237–249.
- ECKERDAL, A. E THUNE, M. 2005. Concepções de programadores Java iniciantes sobre "objeto" e "classe", e variação. teoria. *SIGCSE Bull.* 37, 3, 89–93.
- ECKERDAL, A., THUNE, M., E BERGLUND, A. 2005. O que é necessário para aprender o 'pensamento de programação'? In *Anais do Workshop Internacional sobre Pesquisa em Educação Computacional (ICER'05)*. ACM, 135–142.
- ELLIOTT TEW, A. 2010. Avaliando o conhecimento de conceitos fundamentais de informática introdutória de forma independente da linguagem. Tese de doutorado, Escola de Computação Interativa, Instituto de Tecnologia da Geórgia.
- FITZGERALD, S., LEWANDOWSKI, G., MCCAULEY, R., MURPHY, L., SIMON, B., THOMAS, L., E ZANDER, C. 2008. Depuração: encontrando, corrigindo e errando, um estudo multi-institucional de depuradores novatos. *Ciência da Computação. Educ.* 18, 2, 93–116.
- FLEURY, AE 1991. Passagem de parâmetros: as regras que os alunos constroem. *SIGCSE Bull.* 23, 1, 283–286.
- FLEURY, AE 2000. Programação em Java: regras construídas por estudantes. *SIGCSE Bull.* 32, 1, 197–201.
- GENTNER, D. E GENTNER, DR. 1983. Águas correntes ou multidões fervilhantes: modelos mentais de eletricidade. Em *Modelos Mentais*, D. Gentner e AL Stevens, Eds., Lawrence Erlbaum, 99–130.
- GENTNER, D. E STEVENS, AL. 1983. *Modelos Mentais*. Lawrence Erlbaum.
- GEORGE, CE 2000a. EROSI—visualizando recursão e descobrindo novos erros. *SIGCSE Bull.* 32, 1, 305–309.
- GEORGE, CE 2000b. Experiências com novatos: a importância das representações gráficas no apoio a modelos mentais. In *Anais do 12º Workshop do Grupo de Interesse em Psicologia da Programação (PPIG'00)*. 33–44.
- GOLDMAN, K., GROSS, P., HEEREN, C., HERMAN, G., KACZMARCZYK, L., LOUI, MC, E ZILLES, C. 2008. Identificação de conceitos importantes e difíceis em cursos introdutórios de computação usando um processo Delphi. *SIGCSE Bull.* 40, 1, 256–260.
- GREENING, T. 1999. Forças construtivistas emergentes no ensino de ciência da computação: moldando um novo futuro. Em *Educação em Ciência da Computação no Século XXI*, T. Greening, Ed., Springer, 47–80.
- GREENING, T. E KAY, J. 2001. Editorial. *Comput. Sci. Educa.* 11, 3, 189–202.
- GRIES, D. 2008. Uma abordagem baseada em princípios para o ensino de OO em primeiro lugar. *SIGCSE Bull.* 40, 1, 31–35.
- GOTSCH, T., SANDERS, I., E GALPIN, V. 2003. Modelos mentais de recursão. *SIGCSE Bull.* 35, 1, 346–350.
- HENRIKSEN, P. 2007. Candidatura ao SIGCSE 2007 DC. <http://www.cs.kent.ac.uk/archive/people/staff/ph53/SIGCSE2007DCApplication-PoulHenriksen.html>.
- HOLLAND, S., GRIFFITHS, R., E WOODMAN, M. 1997. Evitando concepções errôneas sobre objetos. *SIGCSE Bull.* 29, 1, 131–134.
- HRISTOVA, M., MISRA, A., RUTTER, M. e MERCURI, R. 2003. Identificação e correção de programação Java. erros para alunos introdutórios de ciência da computação. *SIGCSE Bull.* 35, 1, 153–156.
- HUNDHAUSEN, CD, DOUGLAS, SA, E STASKO, JT 2002. Um meta-estudo da eficácia da visualização de algoritmos. *J. Visual Lang. Comput.* 13, 3, 259–290.
- JOHNSON-LAIRD, PN 1983. *Modelos Mentais: Rumo a uma Ciência Cognitiva da Linguagem, Inferência e Compreensão. Consciência*. Harvard University Press.
- KACZMARCZYK, LC, PETRICK, ER, EAST, JP e HERMAN, GL 2010. Identificando concepções errôneas de programação em estudantes. In *Anais do 41º Simpósio Técnico da ACM sobre Educação em Ciência da Computação (SIGCSE'10)*. ACM, 107–111.
- KAHNEY, H. 1983. O que os programadores novatos sabem sobre recursão? In *Anais da Conferência SIGCHI sobre Fatores Humanos em Sistemas de Computação (CHI'83)*. ACM, 235–239.
- KEMPTON, W. 1986. Duas teorias de controle de calor doméstico. *Cognit. Sci.* 10, 75–90.
- KESSEL, CJ E WICKENS, CD 1982. A transferência de habilidades de detecção de falhas entre monitoramento e controle de sistemas dinâmicos. *Fatores Humanos* 24, 1, 49–60.

- KESSLER, CM E ANDERSON, JR 1986. Aprendendo o fluxo de controle: procedimentos recursivos e iterativos. *Hum. Comput. Interact.* 2, 2, 135–166.
- KLEIN, GA 1999. *Fontes de Poder: Como as Pessoas Tomam Decisões*. MIT Press.
- COLLING, M. 2008. Usando o BlueJ para introduzir a programação. Em *Reflexões sobre o Ensino de Programação: Métodos e Implementações*, J. Bennedsen, ME Caspersen e M. Kölling, Eds., Springer, 98–115.
- KUNKLE, WM 2010. O impacto de diferentes abordagens e linguagens de ensino na aprendizagem de conceitos introdutórios de programação por estudantes. Tese de doutorado. Universidade Drexel.
- LAHTINEN, E., ALA-MUTKA, K., E JARVINEN, H.-M. 2005. Um estudo sobre as dificuldades dos programadores iniciantes. *SIGCSE Bull.* 37, 3, 14–18.
- LAND, R. E MEYER, JHF, EDS. 2008. *Conceitos Limiães nas Disciplinas*. Sense Publishers.
- LAROCHELLE, M., BEDNARZ, N., E GARRISON, J., EDS. 1998. *Construtivismo e Educação*. Cambridge University Press .
- LISTER, R., ADAMS, ES, FITZGERALD, S., FONE, W., HAMER, J., LINDHOLM, M., MCCARTNEY, R., MOSTROM, JE, SANDERS, K., SEPPAL, A, O., SIMON, B., E THOMAS, L. 2004. Um estudo multinacional das habilidades de leitura e rastreamento em programadores iniciantes. *SIGCSE Bull.* 36, 4, 119–150.
- LISTER, R., BERGLUND, A., CLEAR, T., BERGIN, J., GARVIN-DOXAS, K., HANKS, B., HITCHNER, L., LUXTON-REILLY, A., SANDERS, K., SCHULTE, C., E WHALLEY, JL 2006. Perspectivas de pesquisa sobre o debate objeto-inicial. *SIGCSE Bull.* 38, 4, 146–165.
- MA, L. 2007. Investigando e aprimorando os modelos mentais de conceitos de programação de programadores iniciantes. Tese de doutorado, Departamento de Ciência da Computação e Informação, Universidade de Strathclyde.
- MADISON, S. E GIFFORD, J. 1997. Passagem de parâmetros: O construto dos novatos de concepção. Relatório de pesquisa. <http://eric.ed.gov/PDFS/ED406211.pdf>.
- MARKMAN, AB E GENTNER, D. 2001. Pensamento. *Ann. Rev. Psychol.* 52, 223–247.
- MARTON, F. 2000. A estrutura da consciência. Em *Fenomenografia*, JA Bowden e E. Walsh, Eds., RMIT Imprensa Universitária, 102–116.
- MARTON, F. E BOOTH, S. 1997. *Aprendizagem e Consciência*. Lawrence Erlbaum.
- MARTON, F., RUNESSON, U., E TSUI, ABM 2004. O espaço da aprendizagem. Em *Discurso em sala de aula e o Espaço de Aprendizagem*, F. Marton e ABM Tsui, Eds., Lawrence Erlbaum, 3–40.
- MAYER, RE 1976. Algumas condições de aprendizagem significativa para programação de computadores: organizadores prévios e controle do sujeito sobre a ordem dos quadros. *J. Educ. Psychol.* 68, 143–150.
- MAYER, RE 1981. A psicologia de como os novatos aprendem programação de computadores. *ACM Comput. Surv.* 13, 1, 121–141.
- MENAND, L. 1997. *Pragmatismo: Uma coletânea*. Vintage.
- MEYER, JHF E LAND, R. 2003. Conceitos limiães e conhecimento problemático: ligações com modos de pensar e praticar dentro das disciplinas. Em *Melhorando a aprendizagem do aluno — Dez anos depois*, C. Rust, Ed., Oxford Centre for Staff and Learning Development.
- MEYER, JHF E LAND, R., EDS. 2006. *Superando as barreiras à compreensão do aluno: conceitos limiães. e Conhecimento Problemático*. Routledge.
- MILLER, LA 1981. Programação em linguagem natural: estilos, estratégias e contrastes. *IBM Syst. J.* 20, 2, 184–215.
- MILNE, I. E ROWE, G. 2002. Dificuldades no aprendizado e no ensino de programação — visões de alunos e professores. tutores. *Edu. Inf. Technol.* 7, 1, 55–66.
- MORGAN, DL 2007. Paradigmas perdidos e pragmatismo recuperado. *J. Mixed Methods Res.* 1, 1, 48–76.
- MURPHY, L., MCCAULEY, R., e FITZGERALD, S. 2012. Perguntas “explique em linguagem simples”: implicações para o ensino. In *Anais do 43º Simpósio Técnico da ACM sobre Educação em Ciência da Computação (SIGCSE’12)*. ACM, Nova York, 385–390.
- NAPS, TL, ROSSLING, G., ALMSTRUM, V., DANN, W., FLEISCHER, R., HUNDHAUSEN, C., KORHONEN, A., MALMI, L., MCNALLY, M., RODGER, S., E VELAZQUEZ-ITURBIDE, JA 2003. Explorando o papel da visualização e do engajamento no ensino de ciência da computação. *SIGCSE Bull.* 35, 2, 131–152.
- NORMAN, DA 1983. Algumas observações sobre modelos mentais. Em *Modelos Mentais*, D. Gentner e AL Stevens, Eds., Lawrence Erlbaum, 7–14.
- PANE, JF, RATANAMAHATANA, CA, E MYERS, BA 2001. Estudando a linguagem e a estrutura em soluções de não-programadores para problemas de programação. *Int. J. Hum. Comput. Stud.* 54, 2, 237–264.
- PANG, MF 2003. Duas faces da variação: sobre a continuidade no movimento fenomenográfico. *Scand. J. Educ. Res.* 47, 2, 145–156.
- PEA, RD 1986. “Erros” conceituais independentes de linguagem na programação de iniciantes. *J. Educ. Comput. Res.* 2, 1, 25–36.

- PERKINS, D. 2006. Construtivismo e conhecimento problemático. Em *Superando as barreiras à compreensão do aluno: conceitos limiares e conhecimento problemático*, JHF Meyer e R. Land, Eds., Routledge, 33–47.
- PERKINS, DN, HANCOCK, C., HOBBS, R., MARTIN, F., E SIMMONS, R. 1986. Condições de aprendizagem em programadores novatos. *J. Educ. Comput. Res.* 2, 1, 37–55.
- PERKINS, DN, SCHWARTZ, S., E SIMMONS, R. 1990. Estratégias instrucionais para os problemas de programadores iniciantes. Em *Ensino e Aprendizagem de Programação de Computadores: Múltiplas Perspectivas de Pesquisa*, RE Meyer, Ed., Lawrence Erlbaum, 153–178.
- PERLIS, AJ 1982. Epigramas sobre programação. *SIGPLAN Not.* 17, 9, 7–13.
- PHILLIPS, DC 1995. O bom, o mau e o feio: as muitas faces do construtivismo. *Educ. Res.* 24, 7, 5–12.
- PHILLIPS, DC, ED. 2000. *Construtivismo na Educação: Opiniões e Segundas Opiniões sobre Temas Controversos* Questões. Sociedade Nacional para o Estudo da Educação.
- PUTNAM, RT, SLEEMAN, D., BAXTER, JA, e KUSPA, LK 1986. Um resumo de equívocos sobre o ensino médio. Programadores BASIC. *J. Educ. Comput. Res.* 2, 4, 459–72.
- RAGONIS, N. E BEN-ARI, M. 2005a. Uma investigação de longo prazo da compreensão de conceitos de POO por novatos. *Comput. Sci. Educ.* 15, 3, 203–221.
- RAGONIS, N. E BEN-ARI, M. 2005b. Sobre a compreensão da estática e da dinâmica de programas orientados a objetos. *SIGCSE Bull.* 37, 1, 226–330.
- RAMADHAN, HA, DEEK, F., e SHILAB, K. 2001. Incorporando visualização de software no projeto de sistemas inteligentes. sistemas de diagnóstico para programação do usuário. *Artif. Intell. Rev.* 16, 61–84.
- SAJANIEMI, J. E KUITTINEN, M. 2008. Dos procedimentos aos objetos: uma agenda de pesquisa para a psicologia do Programação orientada a objetos na educação. *Hum. Technol.* 4, 1, 75–91.
- SAJANIEMI, J., KUITTINEN, M., E TIKANSALO, T. 2008. Um estudo do desenvolvimento das visualizações do estado do programa pelos alunos durante um curso elementar de programação orientada a objetos. *J. Educ. Res. Comput.* 7, 4, 1–31.
- SAMURÇAY, R. 1989. O conceito de variável em programação: seu significado e uso na resolução de problemas por programadores iniciantes. Em *Estudando o Programador Iniciante*, RE Mayer, Ed., Lawrence Erlbaum Associates, 161–178.
- SAVERY, JR E DUFFY, TM 1995. Aprendizagem baseada em problemas: um modelo instrucional e sua estrutura construtivista. Em *Ambientes de Aprendizagem Construtivistas: Estudos de Caso em Design Instrucional*, B. Wilson, Ed., Educational Technology Publications, 135–150.
- SCHULTE, C. E BENNEDSEN, J. 2006. O que os professores ensinam na programação introdutória? In *Anais do 2º Workshop Internacional sobre Pesquisa em Educação Computacional (ICER'06)*. ACM, 17–28.
- SCHUMACHER, RM 1987. Aquisição de modelos mentais. In *Anais da 4ª Conferência Anual de Ciências Humanas do Centro-Oeste. Conferência Factors/Ergonomics*. Springer, 142–148.
- SCHUMACHER, RM E CZERWINSKI, MP 1992. Modelos mentais e a aquisição de conhecimento especializado. Em *A Psicologia da Expertise: Pesquisa Cognitiva e IA Empírica*, RR Hoffman, Ed., Springer, 61–79.
- SCHUMACHER, RM E GENTNER, D. 1988. Transferência de treinamento como mapeamento analógico. *IEEE Trans. Syst. Homem Cibern.* 18, 4, 592–600.
- SCHWILL, A. 1994. Ideias fundamentais da ciência da computação. *Bull. Eur. Assoc. Theor. Comput. Sci.* 53, 274–274.
- SHINNERS-KENNEDY, D. 2008. A cotidianeidade dos conceitos limiares: o estado como exemplo da ciência da computação. Em *Conceitos Limiares nas Disciplinas*, R. Land e JHF Meyer, Eds., Sense Publishers, 119–128.
- SHMALLO, R., RAGONIS, N., E GINAT, D. 2012. Programação Orientada a Objetos Fuzzy: interpretações de termos expandidos e reduzidos. In *Anais da 17ª Conferência Anual da ACM sobre Inovação e Tecnologia no Ensino de Ciência da Computação (ITICSE'12)*. ACM, Nova York, 309–314.
- SIMON. 2011. Atribuição e sequência: por que alguns alunos não conseguem reconhecer uma simples troca. In *Anais da 11ª Conferência Internacional Koli Calling sobre Pesquisa em Educação Computacional (Koli Calling'11)*. ACM, 16–22.
- SLEEMAN, D., PUTNAM, RT, BAXTER, J., E KUSPA, L. 1986. Pascal e alunos do ensino médio: um estudo de erros. *J. Educ. Comput. Res.* 2, 1, 5–23.
- SMITH, PA e WEBB, GI 1995. Reforçando um modelo genérico de computador para programadores iniciantes. In *Anais da 7ª Conferência da Sociedade Australiana para Computação no Ensino Superior (ASCILITE'95)*.
- SOLOWAY, E. 1986. Aprender a programar = aprender a construir mecanismos e explicações. *Comm. ACM* 29, 9, 850–858.

- SOLOWAY, E., BONAR, J., E EHRLICH, K. 1983. Estratégias cognitivas e construtos de looping: um estudo empírico. *Comm. ACM* 26, 11, 853–860.
- SOLOWAY, E., EHRLICH, K., BONAR, J., e GREENSPAN, J. 1982. O que os novatos sabem sobre programação? Em *Direções em Interações Humano-Computador*, A. Badre e B. Shneiderman, Eds., Ablex Publishing, 27–54.
- SORVA, J. 2007. Compreensões dos alunos sobre o armazenamento de objetos. In *Anais da 7ª Conferência do Mar Báltico sobre Pesquisa em Educação Computacional (Koli Calling'07)*. Sociedade Australiana de Computação, 127–135.
- SORVA, J. 2008. O mesmo, mas diferente — a compreensão dos alunos sobre variáveis primitivas e variáveis de objeto. In *Anais da 8ª Conferência Internacional Koli Calling sobre Pesquisa em Educação Computacional (Koli Calling'08)*. 5–15.
- SORVA, J. 2010. Reflexões sobre conceitos limiares em programação de computadores e além. In *Anais da 10ª Conferência Internacional Koli Calling sobre Pesquisa em Educação Computacional (Koli Calling'10)*. ACM, 21–30.
- SORVA, J. 2012. Simulação visual de programas no ensino introdutório de programação. Tese de doutorado. Departamento de Ciência da Computação e Engenharia, Universidade Aalto.
- SORVA, J., KARAVIRTA, V., e MALMI, L. Uma revisão de sistemas genéricos de visualização de programas para introdução à programação. Educação em programação. *ACM Trans. Comput. Educ.* No prelo.
- STEFFE, LP E GALE, JE, EDS. 1995. *Construtivismo na Educação*. Lawrence Erlbaum.
- STOODLEY, I., CHRISTIE, R. e BRUCE, C. 2004. Experiências de alunos de mestrado aprendendo a programar: um modelo empírico. In *Anais da Conferência Internacional sobre Pesquisa Qualitativa em TI e TI em Pesquisa Qualitativa (QualIT'04)*.
- TASHAKKORI, A. E TEDDLIE, C., EDS. 2010. *Manual Sage de Métodos Mistos em Pesquisa Social e Comportamental*. 2ª ed., Sage.
- TEAGUE, D., CORNEY, M., AHADI, A., e LISTER, R. 2012. Swapping as the “Hello World” of relational reasoning: replications, reflections and extensions. In *Proceedings of the 14th Australasian Conference on Computing Education (ACE'12)*. Australian Computer Society, 87–93.
- TEIF, M. e HAZZAN, O. 2006. Partonomia e taxonomia no pensamento orientado a objetos: ensino fundamental II. Percepções dos alunos sobre conceitos básicos orientados a objetos. *SIGCSE Bull.* 38, 4, 55–60.
- THOTA, N., BERGLUND, A., e CLEAR, T. 2012. Ilustração do pluralismo paradigmático na pesquisa em educação computacional. In *Anais da 14ª Conferência Australiana sobre Educação em Computação (ACE'12)*. Sociedade Australiana de Computação, 103–112.
- THUNE, M. E ECKERDAL, A. 2009. Teoria da variação aplicada às concepções dos alunos sobre programas de computador. *min. Euro. J. Eng. Educ.* 34, 4, 339–347.
- THUNE, M. E ECKERDAL, A. 2010. Concepções dos alunos sobre programação de computadores. Relatório técnico 2010-021, Departamento de Tecnologia da Informação, Universidade de Uppsala.
- TUOVINEN, JE 2000. Otimizando a carga cognitiva do aluno no ensino de informática. In *Anais do Aus-Conferência Transasiática sobre Educação em Computação (ACE'00)*. ACM, 235–241.
- VAGIANOU, E. 2006. Armazenamento de trabalho do programa: um modelo para iniciantes. In *Anais do 6º Congresso do Mar Báltico Conferência sobre Pesquisa em Educação Computacional (Koli Calling'06)*. 69–76.
- VAINIO, V. 2006. Modelos mentais de alunos sobre a execução de programas em um curso básico de programação. Dissertação de mestrado. Departamento de Psicologia, Universidade de Helsinque.
- VAINIO, V. E SAJANIEMI, J. 2007. Fatores que contribuem para a baixa habilidade de rastreamento de programadores iniciantes. *SIGCSE Bull.* 39, 3, 236–240.
- VICTOR, B. 2012. Inventando com base em princípios (vídeo). <http://vimeo.com/36579366> Acessado em fevereiro de 2012.
- WESTBROOK, L. 2006. Modelos mentais: uma visão geral teórica e estudo preliminar. *J. Inf. Sci.* 32, 6, 563–579.
- WIEDENBECK, S. 1989. Aprendendo iteração e recursão a partir de exemplos. *Int. J. Man Mach. Stud.* 30, 1, 1–22.
- WIEDENBECK, S. E RAMALINGAM, V. 1999. Compreensão de programas pequenos escritos em estilos procedurais e orientados a objetos por iniciantes. *Int. J. Hum. Comput. Stud.* 51, 1, 71–87.
- WIEDENBECK, S., RAMALINGAM, V., SARASAMMA, S., E CORRITORE, CL 1999. Uma comparação da compreensão de programas orientados a objetos e procedurais por programadores novatos. *Interact. Comput.* 11, 3, 255–282.
- ZANDER, J., BOUSTEDT, J., ECKERDAL, A., MCCARTNEY, R., MOSTROM, J., JE, RATCLIFFE, M., E SANDERS, K. 2008. Conceitos limiares em ciência da computação: uma investigação empírica multinacional. Em *Conceitos Limiares nas Disciplinas*, R. Land e JHF Meyer, Eds., Sense Publishers, 105–118.

Recebido em maio de 2012; revisado em setembro de 2012; aceito em janeiro de 2013.